100 Exercises to Learn Rust

সম্পূর্ণ প্রশ্ন তালিকা — বিষয় অনুযায়ী বিভাগ

Complete Question Reference — Organized by Topic

Original Book: 100 Exercises to Learn Rust by Luca Palmieri (Mainmatter)
GitHub: github.com/mainmatter/100-exercises-to-learn-rust

📖 নির্দেশাবলী (Instructions)

বাংলা নির্দেশাবলী

এই ডকুমেন্টে "100 Exercises to Learn Rust" বইটির সমস্ত ৯৮টি এক্সারসাইজের প্রশ্ন রয়েছে, বিষয় অনুযায়ী ৮টি বিভাগে সাজানো।

প্রতিটি এক্সারসাইজে রয়েছে: ক্রমিক নম্বর, শিরোনাম (ইংরেজি ও বাংলা), ফাইল অবস্থান, এবং বিস্তারিত বিবরণ (ইংরেজি ও বাংলা)।

প্র্যাকটিক্যাল এক্সারসাইজগুলো করতে হলে GitHub রিপোজিটরি ক্লোন করো এবং সংশ্লিষ্ট ফোল্ডারে কাজ করো।

প্রতিটি এক্সারসাইজের src/lib.rs ফাইলে বিস্তারিত নির্দেশনা এবং TODO মার্কার থাকবে।

সমাধান যাচাই করতে `wr` কমান্ড অথবা `cargo test` ব্যবহার করো।

সমাধান খুঁজে পাওয়া যাবে GitHub রিপোজিটরির solutions branch-এ।

English Instructions

This document contains all 98 exercises from the book "100 Exercises to Learn Rust", organised into 8 sections by topic.

Each exercise includes: a sequential number, title (English & Bengali), file location, and a detailed description (English & Bengali).

To complete the exercises, clone the GitHub repository and work inside the relevant folder.

Each exercise's src/lib.rs file contains detailed instructions and TODO markers to guide your implementation.

Use the `wr` command or `cargo test` to verify your solutions against the automated test suite.

Reference solutions can be found in the `solutions` branch of the GitHub repository.

📚 রেফারেন্স তথ্য (Reference Information)

বইয়ের নাম / **Book Title:** *100 Exercises to Learn Rust*
লেখক / **Author:** *Luca Palmieri (Principal Engineering Consultant, Mainmatter)*
প্রকাশক / **Publisher:** *Mainmatter (https://mainmatter.com)*
GitHub Repository: *https://github.com/mainmatter/100-exercises-to-learn-rust*
Solutions Branch: *https://github.com/mainmatter/100-exercises-to-learn-rust/tree/solutions*
অনলাইন পড়া / **Read Online:** *https://rust-exercises.com*
Amazon Paperback: *Available on Amazon (search '100 Exercises to Learn Rust')*
মোট এক্সারসাইজ / **Total Exercises:** *98 (across 8 chapters)*
প্রয়োজনীয় টুলস / **Required Tools:** *Rust (via rustup), VS Code + rust-analyzer or RustRover IDE*
শুরু করার কমান্ড / **Start Command:** *git clone git@github.com:mainmatter/100-exercises-to-learn-rust.git*

📂 Section 1: Welcome & Syntax (Chapter 1)

অধ্যায় ১: ওয়েলকাম ও সিনট্যাক্স

Exercise 1: Welcome to Rust | এক্সারসাইজ 1: রাস্টে স্বাগতম

📁 **Location:** *exercises/01_intro/00_welcome*

📝 **Task (English):**

Your very first Rust exercise. Open the file src/lib.rs inside the exercise folder. You will find a greeting() function that currently returns a placeholder string. Fix the function so that it returns the exact string "I'm ready to learn Rust!". Run `wr` (the course verification tool) to confirm your solution. This exercise familiarises you with the overall course workflow: read the instructions in src/lib.rs, make your edit, and run the automated test suite to verify your answer.

📝 **কাজ (বাংলা):**

এটি তোমার প্রথম Rust এক্সারসাইজ। exercises/01_intro/00_welcome ফোল্ডারের src/lib.rs ফাইলটি খোলো। greeting() ফাংশনটি এখন একটি placeholder string রিটার্ন করছে। ফাংশনটিকে ঠিক করো যাতে এটি "I'm ready to learn Rust!" স্ট্রিংটি রিটার্ন করে। সমাধান যাচাই করতে `wr` কমান্ড চালাও।

Exercise 2: Rust Syntax Basics | এক্সারসাইজ 2: রাস্টের মৌলিক সিনট্যাক্স

📁 **Location:** *exercises/01_intro/01_syntax*

📝 **Task (English):**

Practice Rust's fundamental syntax elements. You will work with function definitions using the `fn` keyword, function parameters with type annotations (e.g. `name: &str`), return types declared with `->`, single-line comments using `//`, and the unit type `()`. You will also practice returning values — both explicitly using the `return` keyword and implicitly by writing an expression as the last line of a function body (without a semicolon). Rust is a statically typed language, so every value must have a known type at compile time.

📝 **কাজ (বাংলা):**

Rust-এর মৌলিক সিনট্যাক্স উপাদান নিয়ে অনুশীলন করো। `fn` কীওয়ার্ড দিয়ে ফাংশন সংজ্ঞায়িত করা, টাইপ অ্যানোটেশন সহ প্যারামিটার, `->` দিয়ে রিটার্ন টাইপ ঘোষণা, `//` দিয়ে কমেন্ট এবং ইউনিট টাইপ `()` নিয়ে কাজ করবে। Rust একটি statically typed ভাষা, তাই প্রতিটি মানের টাইপ কম্পাইল-টাইমে জানা থাকতে হবে।

📂 Section 2: A Basic Calculator (Chapter 2)

অধ্যায় ২: একটি মৌলিক ক্যালকুলেটর

Exercise 3: Calculator Introduction | এক্সারসাইজ 3: ক্যালকুলেটর পরিচিতি

📁 **Location:** *exercises/02_basic_calculator/00_intro*

📝 **Task (English):**

This is the introductory exercise for the Basic Calculator chapter. Familiarise yourself with the chapter topics: defining and calling functions, declaring and using variables, Rust's primitive integer and boolean types, arithmetic operators (including overflow/underflow behaviour), comparison operators, control flow using `if`/`else`, and panics. Complete this exercise before moving on — it builds the foundation for all subsequent calculator exercises.

📝 **কাজ (বাংলা):**

এটি বেসিক ক্যালকুলেটর অধ্যায়ের ভূমিকামূলক এক্সারসাইজ। ফাংশন সংজ্ঞায়িত ও কল করা, ভেরিয়েবল ডিক্লেয়ার করা, পূর্ণসংখ্যা ও বুলিয়ান টাইপ, অ্যারিথমেটিক অপারেটর (overflow/underflow সহ), তুলনা অপারেটর এবং `if`/`else` ব্যবহার করে কন্ট্রোল ফ্লো সম্পর্কে জানো।

Exercise 4: Integer Types | এক্সারসাইজ 4: পূর্ণসংখ্যার ধরন

📁 **Location:** *exercises/02_basic_calculator/01_integers*

📝 **Task (English):**

Work with Rust's integer types. Rust provides both signed integers (`i8`, `i16`, `i32`, `i64`, `i128`, `isize`) and unsigned integers (`u8`, `u16`, `u32`, `u64`, `u128`, `usize`). The number suffix indicates the bit-width. The `usize` and `isize` types are pointer-sized and depend on the target architecture. Practice computing arithmetic expressions using `u32` and other integer types. Remember that all type conversions in Rust are explicit — the compiler will not automatically convert between integer types of different widths.

📝 **কাজ (বাংলা):**

Rust-এর integer টাইপগুলো নিয়ে কাজ করো। Rust-এ signed integer (`i8`, `i16`, `i32`, `i64`, `i128`, `isize`) এবং unsigned integer (`u8`, `u16`, `u32`, `u64`, `u128`, `usize`) রয়েছে। সংখ্যার suffix bit-width নির্দেশ করে। `u32` এবং অন্যান্য integer টাইপ ব্যবহার করে অ্যারিথমেটিক এক্সপ্রেশন গণনার অনুশীলন করো।

Exercise 5: Variables | এক্সারসাইজ 5: ভেরিয়েবল

📁 **Location:** *exercises/02_basic_calculator/02_variables*

📝 **Task (English):**

Declare and use variables in Rust. Use the `let` keyword to create variable bindings. By default, variables are immutable in Rust; add the `mut` keyword to make a variable mutable. Type annotations are optional when the compiler can infer the type, but are required otherwise. Practice writing a `speed(start, end, time_elapsed)` function that computes distance divided by elapsed time. Also learn that using an uninitialised variable triggers a compile-time error, not a runtime crash — Rust's safety guarantees start at compile time.

📝 **কাজ (বাংলা):**

Rust-এ ভেরিয়েবল ডিক্লেয়ার ও ব্যবহার করো। `let` কীওয়ার্ড দিয়ে ভেরিয়েবল বাইন্ডিং তৈরি করো। Rust-এ ডিফল্টভাবে ভেরিয়েবল immutable; মিউটেবল করতে `mut` কীওয়ার্ড যোগ করো। শুরু না করা ভেরিয়েবল ব্যবহার করলে কম্পাইল-টাইম এরর হয়, রানটাইম ক্র্যাশ নয়।

Exercise 6: Control Flow — if/else | এক্সারসাইজ 6: কন্ট্রোল ফ্লো — if/else

📁 **Location:** *exercises/02_basic_calculator/03_if_else*

📝 **Task (English):**

Write conditional logic in Rust using `if`/`else` expressions. Unlike many languages, `if`/`else` is an expression in Rust — it evaluates to a value, which means it can be used on the right-hand side of a `let` binding. When used this way, both branches must evaluate to the same type. Practice using comparison operators (`>`, `<`, `>=`, `<=`, `==`, `!=`) and logical operators (`&&`, `||`, `!`) inside conditions. Complete the exercise to handle arithmetic operations that require branching based on the input values.

📝 কাজ (বাংলা):

Rust-এ `if`/`else` এক্সপ্রেশন ব্যবহার করে কন্ডিশনাল লজিক লেখো। Rust-এ `if`/`else` একটি expression — এটি একটি মান মূল্যায়ন করে, তাই `let` বাইন্ডিং-এর ডানদিকে ব্যবহার করা যায়। তুলনা অপারেটর (`>`, `<`, `>=`, `<=`, `==`, `!=`) এবং লজিক্যাল অপারেটর (`&&`, `||`, `!`) অনুশীলন করো।

Exercise 7: Panics | এক্সারসাইজ 7: প্যানিক

📁 **Location:** *exercises/02_basic_calculator/04_panics*

📝 **Task (English):**

Learn about Rust's panic mechanism for unrecoverable errors. A panic is triggered when the program encounters a situation it cannot safely continue from — for example, dividing by zero or accessing an array out of bounds. You can intentionally trigger a panic using the `panic!` macro. Your task is to modify the `speed` function from the previous section to explicitly panic with a descriptive error message when `time_elapsed` is zero, rather than causing an implicit divide-by-zero panic. Use `panic!("error message")` to signal the problem clearly.

📝 কাজ (বাংলা):

Rust-এর panic মেকানিজম সম্পর্কে জানো যা অপুনরুদ্ধারযোগ্য ত্রুটির জন্য ব্যবহৃত হয়। `panic!` ম্যাক্রো ব্যবহার করে ইচ্ছাকৃতভাবে panic ট্রিগার করা যায়। `time_elapsed` শূন্য হলে একটি বর্ণনামূলক এরর মেসেজ সহ `panic!` ব্যবহার করো।

Exercise 8: Factorial (Recursion) | এক্সারসাইজ 8: ফ্যাক্টোরিয়াল (পুনরাবৃত্তি)

📁 **Location:** *exercises/02_basic_calculator/05_factorial*

📝 **Task (English):**

Implement the factorial function using recursion. Given the concepts covered so far — functions, integer types, arithmetic operators, `if`/`else` expressions — implement `factorial(n: u32) -> u32` that returns n! (n factorial). Recall that 0! = 1, 1! = 1, and n! = n × (n-1)! for n > 1. This is a classic recursive algorithm that will test your understanding of function calls, return values, and conditional branching in Rust.

📝 কাজ (বাংলা):

পুনরাবৃত্তি (recursion) ব্যবহার করে factorial ফাংশন ইমপ্লিমেন্ট করো। `factorial(n: u32) -> u32` ফাংশন লেখো যা n! (n ফ্যাক্টোরিয়াল) রিটার্ন করে। মনে রাখো: 0! = 1, 1! = 1, এবং n > 1 হলে n! = n × (n-1)!।

Exercise 9: Loops — while | এক্সারসাইজ 9: লুপ — while

📁 **Location:** *exercises/02_basic_calculator/06_while*

📝 **Task (English):**

Re-implement the factorial function using a `while` loop instead of recursion. A `while` loop executes its body repeatedly as long as the given condition evaluates to `true`. Syntax: `while <condition> { /* body */ }`. Practice using a mutable accumulator variable (`let mut result = 1u64`) and a loop counter to iteratively compute the factorial. Loops are useful when you need to repeat a block of code a variable number of times based on a runtime condition.

📝 **কাজ (বাংলা):**

`while` লুপ ব্যবহার করে factorial ফাংশন পুনরায় ইমপ্লিমেন্ট করো। `while` লুপ শর্ত সত্য থাকা পর্যন্ত বারবার চলে। একটি mutable accumulator ভেরিয়েবল এবং লুপ কাউন্টার ব্যবহার করে iteratively factorial গণনার অনুশীলন করো।

Exercise 10: Loops — for | এক্সারসাইজ 10: লুপ — for

📁 **Location:** *exercises/02_basic_calculator/07_for*

📝 **Task (English):**

Re-implement the factorial function once more, this time using a `for` loop with a range expression. Range syntax: `for i in 1..=n { ... }` iterates over the inclusive range [1, n]. The `..` operator creates an exclusive range and `..=` creates an inclusive range. For loops in Rust are actually iterators under the hood — any type implementing the `IntoIterator` trait can be used in a `for` loop. Practice this more idiomatic Rust style for bounded iteration.

📝 **কাজ (বাংলা):**

`for` লুপ ও range expression ব্যবহার করে factorial ফাংশন আবার ইমপ্লিমেন্ট করো। Range সিনট্যাক্স: `for i in 1..=n { ... }` inclusive range [1, n] iterate করে। `..` exclusive range এবং `..=` inclusive range তৈরি করে।

Exercise 11: Integer Overflow | এক্সারসাইজ 11: পূর্ণসংখ্যার উপচে পড়া (Overflow)

📁 **Location:** *exercises/02_basic_calculator/08_overflow*

📝 **Task (English):**

Understand how Rust handles integer overflow. In debug builds, Rust will panic at runtime when integer arithmetic overflows (e.g. `255u8 + 1`). In release builds (compiled with `--release`), the same operation wraps around silently. You can opt into specific overflow behaviours explicitly using the wrapping methods (`wrapping_add`, `wrapping_sub`, `wrapping_mul`), the checked methods (returning `Option`), or the saturating/overflowing methods. Modify a function to handle potential overflow by choosing the appropriate method.

📝 **কাজ (বাংলা):**

Rust কীভাবে integer overflow সামলায় তা বোঝো। Debug build-এ overflow হলে runtime panic হয়। Release build-এ (`--release`) একই অপারেশন silently wrap করে। `wrapping_add`, `wrapping_sub`, `wrapping_mul` বা checked/saturating মেথড ব্যবহার করে overflow সামলাও।

Exercise 12: Saturating Arithmetic | এক্সারসাইজ 12: স্যাচুরেটিং অ্যারিথমেটিক

📁 **Location:** *exercises/02_basic_calculator/09_saturating*

📝 **Task (English):**

Use saturating arithmetic to handle edge cases gracefully. Saturating operations clamp the result to the type's maximum or minimum value rather than wrapping or panicking. For example, `255u8.saturating_add(1)` returns `255` (not 0 or a panic). Similarly, `0u8.saturating_sub(1)` returns `0`. The standard library provides `saturating_add`, `saturating_sub`, `saturating_mul`, and

`saturating_pow` methods on all integer types. Implement a function using saturating arithmetic to safely compute results without risk of overflow.

📝 কাজ (বাংলা):

Edge case গুলো সুন্দরভাবে সামলাতে saturating arithmetic ব্যবহার করো। Saturating অপারেশন ফলাফলকে টাইপের সর্বোচ্চ বা সর্বনিম্ন মানে ক্ল্যাম্প করে। উদাহরণ: `255u8.saturating_add(1)` রিটার্ন করে `255`। `saturating_add`, `saturating_sub`, `saturating_mul` মেথড ব্যবহার করো।

Exercise 13: Type Casting with as | এক্সারসাইজ 13: `as` দিয়ে টাইপ কাস্টিং

📁 **Location:** *exercises/02_basic_calculator/10_as_casting*

📝 **Task (English):**

Learn to convert between numeric types using Rust's `as` casting operator. The `as` keyword performs primitive type conversions: `let x: i32 = 5; let y = x as u32;`. When casting from a larger type to a smaller one, the value is truncated (the high bits are dropped). When casting from a signed type to unsigned, the bit pattern is preserved and reinterpreted. Use `as` carefully because it can silently lose information. The exercise requires you to apply `as` casting to bridge between two integer types in a calculation.

📝 কাজ (বাংলা):

Rust-এর `as` কাস্টিং অপারেটর ব্যবহার করে numeric টাইপের মধ্যে রূপান্তর করতে শেখো। `let x: i32 = 5; let y = x as u32;` এর মতো লেখো। বড় থেকে ছোট টাইপে কাস্ট করলে মান truncate হয়। `as` সাবধানে ব্যবহার করো কারণ এটি নিঃশব্দে তথ্য হারাতে পারে।

📂 Section 3: Modelling a Ticket — Part 1 (Chapter 3)

অধ্যায় ৩: একটি টিকেট মডেলিং — ভাগ ১

Exercise 14: Ticket Introduction | এক্সারসাইজ 14: টিকেট পরিচিতি

📁 **Location:** *exercises/03_ticket_v1/00_intro*

📝 **Task (English):**

This is the intro to the Ticket modelling chapter, which covers structs, ownership, references and borrowing, and memory management (stack vs. heap). You will build a simple ticket-tracking data structure similar to a project-management task. The exercise asks you to set up the initial project structure and understand the topics to be covered: custom types with `struct`, Rust's ownership rules, references, and how memory is allocated and freed.

📝 কাজ (বাংলা):

এটি টিকেট মডেলিং অধ্যায়ের ভূমিকা, যা struct, ownership, references এবং borrowing, এবং মেমরি ম্যানেজমেন্ট (stack বনাম heap) কভার করে। একটি simple ticket-tracking ডেটা স্ট্রাকচার তৈরি করো।

Exercise 15: Structs | এক্সারসাইজ 15: স্ট্রাক্ট

📁 **Location:** *exercises/03_ticket_v1/01_struct*

📝 **Task (English):**

Define a Rust struct to model a support ticket. A struct is a custom data type that groups related fields together. Define a `Ticket` struct with three fields: `title: String`, `description: String`, and `status: String`. Then implement a `Ticket::new` associated function (constructor) that creates a new `Ticket` instance from provided values. Practice the field access syntax (`ticket.title`) and the method call syntax (`ticket.method()`). Remember that `impl` blocks are used to attach methods and associated functions to a struct.

📝 কাজ (বাংলা):

একটি support ticket মডেল করতে Rust struct সংজ্ঞায়িত করো। `title: String`, `description: String`, এবং `status: String` তিনটি ফিল্ড সহ একটি `Ticket` struct তৈরি করো। তারপর `Ticket::new` associated function (constructor) ইমপ্লিমেন্ট করো।

Exercise 16: Validation | এক্সারসাইজ 16: বৈধতা যাচাই (Validation)

📁 **Location:** *exercises/03_ticket_v1/02_validation*

📝 **Task (English):**

Add input validation to the `Ticket::new` constructor. Currently any string can be passed as a title, description, or status. Improve this by panicking when invalid values are supplied: the title must not be empty and must not exceed 50 characters; the description must not be empty and must not exceed 500 characters; the status must be one of "To-Do", "In Progress", or "Done". Use `String` methods such as `.is_empty()`, `.len()`, and comparison operators. Check the `String` documentation for a full list of available methods.

📝 কাজ (বাংলা):

`Ticket::new` কনস্ট্রাক্টরে input validation যোগ করো। title খালি হওয়া চলবে না ও ৫০ অক্ষরের বেশি হওয়া চলবে না; description খালি হওয়া চলবে না ও ৫০০ অক্ষরের বেশি হওয়া চলবে না; status শুধুমাত্র "To-Do", "In Progress", বা "Done" হতে পারবে।

Exercise 17: Modules | এক্সারসাইজ 17: মডিউল

📁 **Location:** *exercises/03_ticket_v1/03_modules*

📝 **Task (English):**

Organise your code into Rust modules. A module is declared with the `mod` keyword and creates a new namespace. You can have inline modules (`mod my_module { ... }`) or file-based modules (a file named `my_module.rs` or a directory `my_module/mod.rs`). Use `use` to bring items into scope and `super::` to refer to the parent module. The exercise asks you to split the ticket-related code into a separate module and properly expose the types and functions that need to be public. Visualise the module tree to check your structure.

📝 কাজ (বাংলা):

Rust মডিউল ব্যবহার করে কোড সংগঠিত করো। `mod` কীওয়ার্ড দিয়ে মডিউল ঘোষণা করো, যা একটি নতুন namespace তৈরি করে। `use` দিয়ে items scope-এ আনো এবং `super::` দিয়ে parent মডিউলে রেফার করো। টিকেট-সম্পর্কিত কোড একটি পৃথক মডিউলে বিভক্ত করো।

Exercise 18: Visibility | এক্সারসাইজ 18: দৃশ্যমানতা (Visibility)

📁 **Location:** *exercises/03_ticket_v1/04_visibility*

📝 **Task (English):**

Control what is accessible from outside a module using Rust's visibility modifiers. By default, all items in Rust are private to the module they are defined in. Use `pub` to make an item public (accessible from anywhere), `pub(crate)` to restrict access to within the same crate, and

`pub(super)` to allow access from the parent module only. Apply the appropriate visibility modifiers to the `Ticket` struct and its fields so that the struct itself is public but certain fields remain private, enforcing encapsulation.

📝 কাজ (বাংলা):

Rust-এর visibility modifier ব্যবহার করে মডিউলের বাইরে থেকে কী অ্যাক্সেসযোগ্য তা নিয়ন্ত্রণ করো। ডিফল্টভাবে সব item private। `pub` দিয়ে public করো, `pub(crate)` দিয়ে একই crate-এ সীমাবদ্ধ করো, এবং `pub(super)` দিয়ে parent মডিউলে সীমাবদ্ধ করো।

Exercise 19: Encapsulation | এক্সারসাইজ 19: এনক্যাপসুলেশন

📁 **Location:** *exercises/03_ticket_v1/05_encapsulation*

📝 **Task (English):**

Achieve proper encapsulation by making struct fields private and exposing only accessor (getter) methods. Once a field is private, code outside the module cannot read or modify it directly. Add public getter methods `title(&self) -> &str`, `description(&self) -> &str`, and `status(&self) -> &str` to the `Ticket` struct. These methods borrow `self` immutably (using `&self`) and return a reference to the field value. This pattern prevents external code from reading internal state directly and is the standard Rust approach to encapsulation.

📝 কাজ (বাংলা):

struct field গুলো private করে এবং শুধুমাত্র accessor (getter) মেথড expose করে সঠিক encapsulation অর্জন করো। `title(&self) -> &str`, `description(&self) -> &str`, এবং `status(&self) -> &str` পাবলিক getter মেথড যোগ করো।

Exercise 20: Ownership | এক্সারসাইজ 20: মালিকানা (Ownership)

📁 **Location:** *exercises/03_ticket_v1/06_ownership*

📝 **Task (English):**

Understand Rust's ownership system — the foundation of its memory safety guarantees. Every value in Rust has exactly one owner at any time. When the owner goes out of scope, the value is dropped (memory freed). Ownership can be transferred (moved) to another variable or function. Once a value has been moved, the original variable can no longer be used. Practice ownership transfer by working with `String` values and understanding when the compiler reports "value used after move" errors. Use borrowing (references) when you only need temporary access without transferring ownership.

📝 কাজ (বাংলা):

Rust-এর ownership সিস্টেম বোঝো — এটি Rust-এর মেমরি সেফটি গ্যারান্টির ভিত্তি। প্রতিটি মানের ঠিক একজন owner থাকে। owner scope-এর বাইরে গেলে মান drop হয়। ownership transfer (move) করা যায়। `String` মান নিয়ে কাজ করে ownership transfer অনুশীলন করো।

Exercise 21: Mutable References / Setters | এক্সারসাইজ 21: মিউটেবল রেফারেন্স / সেটার

📁 **Location:** *exercises/03_ticket_v1/07_setters*

📝 **Task (English):**

Add setter methods to `Ticket` that allow callers to modify field values after construction. There are two approaches: take `&mut self` (mutable reference to self) and mutate the field in place, or take `self` (ownership), update the field, and return the modified struct. Implement `set_title`, `set_description`, and `set_status` methods. When you take `&mut self`, at most one mutable reference to a value may exist at any time (the borrow checker enforces this). Methods that take `self` and return `Self` can be chained: `ticket.set_title(...).set_description(...)...`.

📝 কাজ (বাংলা):

`Ticket`-এ setter মেথড যোগ করো যা কলারদের তৈরির পরে field মান পরিবর্তন করতে দেয়। `&mut self` নিয়ে in-place পরিবর্তন করার পদ্ধতি বা `self` ownership নিয়ে পরিবর্তন করে struct রিটার্ন করার পদ্ধতি — দুটি approach আছে। `set_title`, `set_description`, এবং `set_status` মেথড ইমপ্লিমেন্ট করো।

Exercise 22: Stack Memory Layout | এক্সারসাইজ 22: স্ট্যাক মেমরি লেআউট

📁 **Location:** *exercises/03_ticket_v1/08_stack*

📝 **Task (English):**

Explore how values are laid out on the stack. The stack is a region of memory used for local variables and function call frames; it operates in a last-in, first-out (LIFO) manner. Stack-allocated values have a fixed, compile-time-known size. For example, `u8` is 1 byte, `u32` is 4 bytes, and `bool` is 1 byte. Use `std::mem::size_of::<T>()` to determine the size of a type. Practice printing the sizes of various types and understanding how struct fields contribute to the overall struct size.

📝 কাজ (বাংলা):

মানগুলো stack-এ কীভাবে সাজানো থাকে তা অন্বেষণ করো। Stack হলো local variable ও function call frame-এর জন্য ব্যবহৃত মেমরির একটি অংশ। `std::mem::size_of::<T>()` ব্যবহার করে টাইপের আকার নির্ধারণ করো।

Exercise 23: Heap Allocation | এক্সারসাইজ 23: হিপ বরাদ্দ (Heap Allocation)

📁 **Location:** *exercises/03_ticket_v1/09_heap*

📝 **Task (English):**

Understand heap memory allocation in Rust. The heap is used for data whose size is not known at compile time or which needs to live longer than the current function call. `String` is a heap-allocated type: it stores its character data on the heap and keeps a pointer, length, and capacity on the stack. When a `String` goes out of scope, its `Drop` implementation automatically frees the heap memory. Explore how `String` is implemented internally using `Vec<u8>`, and understand why the heap allocation is not considered part of the `String`'s stack footprint.

📝 কাজ (বাংলা):

Rust-এ heap মেমরি allocation বোঝো। Heap ব্যবহার হয় যখন কম্পাইল সময়ে ডেটার আকার জানা নেই বা ডেটা বর্তমান ফাংশন কলের চেয়ে দীর্ঘস্থায়ী হতে হবে। `String` একটি heap-allocated টাইপ।

Exercise 24: References in Memory | এক্সারসাইজ 24: মেমরিতে রেফারেন্স

📁 **Location:** *exercises/03_ticket_v1/10_references_in_memory*

📝 **Task (English):**

Understand how references are represented in memory. A reference (`&T`) is a pointer — a value on the stack that stores the memory address of another value. References can point to stack or heap memory. The size of any reference is always `std::mem::size_of::<usize>()` bytes — one pointer-sized word — regardless of the size of the data being referenced. Practice using `std::mem::size_of_val()` and `std::mem::size_of::<T>()` to compare the sizes of owned values versus references to those values.

📝 কাজ (বাংলা):

রেফারেন্সগুলো মেমরিতে কীভাবে উপস্থাপিত হয় তা বোঝো। একটি রেফারেন্স (`&T`) হলো একটি pointer — stack-এ একটি মান যা অন্য মানের মেমরি ঠিকানা সংরক্ষণ করে। যেকোনো রেফারেন্সের আকার সবসময় `usize` bytes।

Exercise 25: Destructors | এক্সারসাইজ 25: ডেস্ট্রাক্টর

📁 **Location:** *exercises/03_ticket_v1/11_destructor*

📝 **Task (English):**

Understand how Rust automatically cleans up resources through destructors. When a value goes out of scope, Rust calls its destructor — the `drop` function defined by the `Drop` trait — to release any resources it owns. This is RAII (Resource Acquisition Is Initialisation) in action. You can also call `std::mem::drop(value)` explicitly to drop a value early. Importantly, you cannot drop a value while an active reference to it exists — the borrow checker prevents use-after-free bugs at compile time. Practice implementing `Drop` for a custom type.

📝 কাজ (বাংলা):

Rust কীভাবে destructor-এর মাধ্যমে স্বয়ংক্রিয়ভাবে resource পরিষ্কার করে তা বোঝো। একটি মান scope-এর বাইরে গেলে, Rust তার destructor — `Drop` trait দ্বারা সংজ্ঞায়িত `drop` ফাংশন — কল করে। একটি কাস্টম টাইপের জন্য `Drop` ইমপ্লিমেন্ট করো।

Exercise 26: Ticket — Wrapping Up | এক্সারসাইজ 26: টিকেট — সমাপ্তি

📁 **Location:** *exercises/03_ticket_v1/12_outro*

📝 **Task (English):**

A consolidation exercise covering all the concepts from Chapter 3: structs, visibility, encapsulation, ownership, mutable references, stack vs. heap, and destructors. You will receive minimal guidance — the goal is to combine everything you have learned to build a complete, well-encapsulated `Ticket` type with a constructor, getters, setters, and validation. This exercise tests your ability to reason about ownership and borrowing in a realistic scenario.

📝 কাজ (বাংলা):

অধ্যায় ৩-এর সমস্ত ধারণা কভার করে একটি consolidation exercise: struct, visibility, encapsulation, ownership, mutable references, stack বনাম heap, এবং destructor। সম্পূর্ণ, সুসংহত `Ticket` টাইপ তৈরি করো।

📂 Section 4: Traits (Chapter 4)

অধ্যায় ৪: ট্রেইট

Exercise 27: Traits — Introduction | এক্সারসাইজ 27: ট্রেইট — ভূমিকা

📁 **Location:** *exercises/04_traits/00_intro*

📝 **Task (English):**

Introduction to Rust's trait system. A trait defines a set of methods that a type must implement to satisfy the trait contract. Traits are Rust's mechanism for abstraction and polymorphism, similar to interfaces in Java or C#. This chapter will cover: defining traits, implementing traits for existing and custom types, operator overloading via traits, derive macros, trait bounds in generic functions, string slices, the `Deref` trait, the `Sized` marker trait, the `From`/`Into` conversion traits, associated types vs. generic parameters, and the `Clone`, `Copy`, and `Drop` traits.

📝 কাজ (বাংলা):

Rust-এর trait সিস্টেমের ভূমিকা। একটি trait মেথডের একটি সেট সংজ্ঞায়িত করে যা একটি টাইপকে অবশ্যই ইমপ্লিমেন্ট করতে হবে। Trait হলো Rust-এর abstraction ও polymorphism-এর মেকানিজম, Java বা C#-এর interface-এর মতো।

Exercise 28: Defining and Implementing Traits | এক্সারসাইজ 28: ট্রেইট সংজ্ঞায়িত ও ইমপ্লিমেন্ট করা

📁 **Location:** *exercises/04_traits/01_trait*

📝 **Task (English):**

Define a custom trait and implement it for one or more types. A trait is declared with the `trait` keyword followed by the trait name and a block containing method signatures. An implementation is provided using `impl TraitName for TypeName { ... }`. Practice defining a `Greet` trait (or similar) with a method `greeting(&self) -> String`, and implementing it for a custom struct. Learn the difference between a required method (no default body) and a provided method (default implementation that can be overridden). Trait methods can be called just like regular methods.

📝 **কাজ (বাংলা):**

একটি কাস্টম trait সংজ্ঞায়িত করো এবং এক বা একাধিক টাইপের জন্য ইমপ্লিমেন্ট করো। `trait` কীওয়ার্ড দিয়ে trait ঘোষণা করো। একটি `Greet` trait (বা অনুরূপ) সংজ্ঞায়িত করো এবং একটি কাস্টম struct-এর জন্য ইমপ্লিমেন্ট করো।

Exercise 29: The Orphan Rule | এক্সারসাইজ 29: অরফান নিয়ম

📁 **Location:** *exercises/04_traits/02_orphan_rule*

📝 **Task (English):**

Learn about Rust's orphan rule, which governs where trait implementations may be written. The rule states: you can implement a trait for a type only if either the trait or the type is defined in the current crate. You cannot implement a foreign trait for a foreign type (a combination of traits and types both defined in external crates). This rule prevents conflicting implementations across crates. Practice working within the orphan rule constraints and understand why it exists — to ensure global coherence of trait implementations.

📝 **কাজ (বাংলা):**

Rust-এর orphan rule সম্পর্কে জানো, যা নির্ধারণ করে trait implementation কোথায় লেখা যায়। নিয়ম: শুধুমাত্র তখনই একটি টাইপের জন্য trait ইমপ্লিমেন্ট করা যাবে যদি trait বা টাইপ বর্তমান crate-এ সংজ্ঞায়িত হয়।

Exercise 30: Operator Overloading | এক্সারসাইজ 30: অপারেটর ওভারলোডিং

📁 **Location:** *exercises/04_traits/03_operator_overloading*

📝 **Task (English):**

Implement operator overloading in Rust using standard library traits. Rust's arithmetic and comparison operators correspond to traits in `std::ops` and `std::cmp`. For example, `+` is `std::ops::Add`, `-` is `std::ops::Sub`, `==` is `std::cmp::PartialEq`, `<` is `std::cmp::PartialOrd`. Implement the `Add` trait for a custom type (such as a 2D point or a vector) so that the `+` operator works on instances of that type. Note that `PartialEq` provides a default `ne` implementation; you only need to implement `eq`.

📝 **কাজ (বাংলা):**

Standard library trait ব্যবহার করে Rust-এ operator overloading ইমপ্লিমেন্ট করো। `+` হলো `std::ops::Add`, `-` হলো `std::ops::Sub`, `==` হলো `std::cmp::PartialEq`। একটি কাস্টম টাইপের জন্য `Add` trait ইমপ্লিমেন্ট করো।

Exercise 31: Derive Macros | এক্সারসাইজ 31: ডেরাইভ ম্যাক্রো

📁 **Location:** *exercises/04_traits/04_derive*

📝 **Task (English):**

Use derive macros to automatically generate trait implementations. The `#[derive(...)]` attribute instructs the Rust compiler to automatically generate an implementation of the listed traits. Commonly derived traits include: `Debug` (enables `{:?}` formatting), `PartialEq` / `Eq` (equality comparisons), `Clone` (cloning a value), `Copy` (stack-copyable values), `Hash` (hashing for HashMap keys), and `PartialOrd` / `Ord` (ordering). Annotate your `Ticket` struct with the appropriate derive attributes and verify that the auto-generated implementations work correctly.

📝 কাজ (বাংলা):

Trait implementation স্বয়ংক্রিয়ভাবে তৈরি করতে derive macro ব্যবহার করো। `#[derive(...)]` অ্যাট্রিবিউট listed trait-এর implementation generate করে। `Debug`, `PartialEq`, `Eq`, `Clone`, `Copy`, `Hash`, `PartialOrd`, `Ord` সাধারণভাবে derive করা হয়।

Exercise 32: Trait Bounds | এক্সারসাইজ 32: ট্রেইট বাউন্ড

📁 **Location:** *exercises/04_traits/05_trait_bounds*

📝 **Task (English):**

Write generic functions with trait bounds to constrain the types they accept. A trait bound specifies that a type parameter must implement a given trait. Syntax: `fn my_func<T: TraitName>(value: T) { ... }` or using a `where` clause: `fn my_func<T>(value: T) where T: TraitName { ... }`. Multiple bounds can be combined with `+`. The function signature is a contract: the compiler verifies at the call site that the concrete type satisfies all bounds. Practice writing a generic print function that requires `Debug` or `Display`, and a generic max function requiring `PartialOrd`.

📝 কাজ (বাংলা):

Generic ফাংশনে trait bound ব্যবহার করো যাতে কোন টাইপ গ্রহণ করা হবে তা সীমাবদ্ধ করা যায়। Syntax: `fn my_func<T: TraitName>(value: T)` বা `where` clause ব্যবহার করো। একাধিক bound `+` দিয়ে মেলানো যায়।

Exercise 33: String Slices (&str) | এক্সারসাইজ 33: স্ট্রিং স্লাইস (&str)

📁 **Location:** *exercises/04_traits/06_str_slice*

📝 **Task (English):**

Understand the relationship between `String` and `&str`. `String` is an owned, heap-allocated string. `&str` is a borrowed string slice — a reference to a contiguous sequence of UTF-8 bytes somewhere in memory (heap, stack, or binary). Functions that only need to read a string should accept `&str` rather than `&String`, because `&String` automatically coerces to `&str` (via `Deref`) but not vice versa. Modify your getter methods to return `&str` instead of `&String` and update the function signatures for maximum flexibility.

📝 কাজ (বাংলা):

`String` এবং `&str`-এর মধ্যে সম্পর্ক বোঝো। `String` একটি owned, heap-allocated string। `&str` একটি borrowed string slice। শুধুমাত্র string read করতে হলে `&String`-এর পরিবর্তে `&str` গ্রহণ করো।

Exercise 34: The Deref Trait | এক্সারসাইজ 34: ডেরেফ ট্রেইট

📁 **Location:** *exercises/04_traits/07_deref*

📝 **Task (English):**

Learn about the `Deref` trait and Rust's automatic deref coercion. `Deref` allows a type to behave like a reference to another type. `String` implements `Deref<Target = str>`, which means `&String` automatically coerces to `&str` in many contexts. Similarly, `Vec<T>` implements `Deref<Target = [T]>`, so `&Vec<T>` coerces to `&[T]`. Deref coercions are applied automatically by the compiler

when passing arguments to functions, making code more ergonomic. Understand when deref coercion applies and be aware it can sometimes make code harder to understand.

📝 কাজ (বাংলা):

`Deref` trait এবং Rust-এর স্বয়ংক্রিয় deref coercion সম্পর্কে জানো। `String` `Deref<Target = str>` ইমপ্লিমেন্ট করে, তাই `&String` অনেক context-এ স্বয়ংক্রিয়ভাবে `&str`-এ coerce হয়।

Exercise 35: The Sized Trait | এক্সারসাইজ 35: সাইজড ট্রেইট

📁 **Location:** *exercises/04_traits/08_sized*

📝 **Task (English):**

Explore the `Sized` marker trait and dynamically sized types (DSTs). `Sized` is automatically implemented for all types whose size is known at compile time. Most types are `Sized`. Types like `str` and `[T]` are not `Sized` — they are dynamically sized types (DSTs). DSTs can only be used behind a reference or pointer (`&str`, `Box<str>`, `&[T]`, etc.). The `?Sized` bound is used to relax the default `Sized` requirement on a generic type parameter, allowing DSTs to be passed. Understand why this matters for generic programming.

📝 কাজ (বাংলা):

`Sized` marker trait এবং dynamically sized types (DSTs) অন্বেষণ করো। `Sized` কম্পাইল সময়ে জানা আকারের সমস্ত টাইপের জন্য স্বয়ংক্রিয়ভাবে ইমপ্লিমেন্ট হয়। `str` এবং `[T]` DST, শুধুমাত্র reference বা pointer-এর পিছনে ব্যবহার করা যায়।

Exercise 36: From and Into | এক্সারসাইজ 36: From এবং Into

📁 **Location:** *exercises/04_traits/09_from*

📝 **Task (English):**

Use the `From` and `Into` conversion traits for infallible type conversions. `From<T>` defines how to create `Self` from `T`. Once you implement `From<T>` for a type, `Into<Self> for T` is automatically derived (they are dual traits). The `.into()` method works when the compiler can infer the target type. Use `From` to enable ergonomic conversions: `String::from("hello")`, `u64::from(42u32)`. Implement `From<&str>` for your `Ticket` type or a related type, and understand why `From` is preferred over custom conversion functions.

📝 কাজ (বাংলা):

Infallible type conversion-এর জন্য `From` এবং `Into` conversion trait ব্যবহার করো। `From<T>` সংজ্ঞায়িত করে কীভাবে `T` থেকে `Self` তৈরি করা যায়। `From<T>` ইমপ্লিমেন্ট করলে `Into<Self> for T` স্বয়ংক্রিয়ভাবে derived হয়।

Exercise 37: Generics vs. Associated Types | এক্সারসাইজ 37: জেনেরিক্স বনাম অ্যাসোসিয়েটেড টাইপ

📁 **Location:** *exercises/04_traits/10_assoc_vs_generic*

📝 **Task (English):**

Understand when to use associated types versus generic type parameters in traits. An associated type (`type Output;` inside a trait) binds a concrete type to each implementation, so a given type can only implement the trait once. A generic parameter (`trait Add<Rhs>`) allows multiple implementations with different type arguments. Use associated types when the relationship is one-to-one (e.g. the output of adding two values of the same type). Use generic parameters when a type should be able to interoperate with multiple other types (e.g. adding a `Point` to a `Vector`).

📝 কাজ (বাংলা):

Trait-এ associated type বনাম generic type parameter কখন ব্যবহার করবে তা বোঝো। Associated type (`type Output;`) প্রতিটি implementation-এ একটি concrete type bind করে। Generic parameter একই trait-এর বিভিন্ন type argument-এর সাথে একাধিক implementation-এর সুযোগ দেয়।

Exercise 38: The Clone Trait | এক্সারসাইজ 38: ক্লোন ট্রেইট

📁 **Location:** *exercises/04_traits/11_clone*

📝 **Task (English):**

Use the `Clone` trait to create explicit copies of values. Calling `.clone()` on a value produces a new value that is a deep copy of the original. Unlike `Copy`, cloning is an explicit, potentially expensive operation (it may allocate heap memory). You can derive `Clone` automatically if all fields of a struct implement `Clone`. Implement `Clone` manually for a type where the derived implementation would not be appropriate. Practice cloning `String` and custom types and understand the difference between a shallow copy (raw bit copy) and a deep copy (cloning heap data).

📝 **কাজ (বাংলা):**

মানের স্পষ্ট কপি তৈরি করতে `Clone` trait ব্যবহার করো। `.clone()` কল করলে original-এর একটি deep copy তৈরি হয়। সব field `Clone` ইমপ্লিমেন্ট করলে `Clone` automatically derive করা যায়।

Exercise 39: The Copy Trait | এক্সারসাইজ 39: কপি ট্রেইট

📁 **Location:** *exercises/04_traits/12_copy*

📝 **Task (English):**

Understand the `Copy` marker trait and how it affects move semantics. Types that implement `Copy` are copied implicitly (bitwise) when assigned or passed to a function, rather than moved. All primitive types (`u32`, `bool`, `f64`, etc.) are `Copy`. A type can be `Copy` only if all its fields are `Copy` and it does not implement `Drop`. Derive `Copy` (along with `Clone`, since `Copy` requires it) for your struct with `#[derive(Copy, Clone)]`. Verify that after `let b = a;`, the variable `a` is still usable because it was copied rather than moved.

📝 **কাজ (বাংলা):**

`Copy` marker trait এবং এটি move semantics-কে কীভাবে প্রভাবিত করে তা বোঝো। `Copy` ইমপ্লিমেন্ট করা টাইপগুলো assign বা ফাংশনে pass করার সময় implicitly (bitwise) copy হয়। `#[derive(Copy, Clone)]` দিয়ে তোমার struct-এ `Copy` যোগ করো।

Exercise 40: The Drop Trait | এক্সারসাইজ 40: ড্রপ ট্রেইট

📁 **Location:** *exercises/04_traits/13_drop*

📝 **Task (English):**

Implement the `Drop` trait to run custom cleanup logic when a value is destroyed. The `Drop` trait has a single method: `fn drop(&mut self)`. You cannot implement both `Copy` and `Drop` for the same type — they are mutually exclusive. Practice implementing `Drop` for a resource-managing type (e.g. a type that wraps a file handle or a network connection) to print a message or release a resource when the value is dropped. Understand that `drop` is called automatically by the runtime and cannot be called explicitly; use `std::mem::drop(value)` to force early dropping.

📝 **কাজ (বাংলা):**

একটি মান ধ্বংস হলে custom cleanup logic চালাতে `Drop` trait ইমপ্লিমেন্ট করো। `Drop` trait-এ একটিমাত্র মেথড আছে: `fn drop(&mut self)`। `Copy` এবং `Drop` একই টাইপের জন্য একসাথে ইমপ্লিমেন্ট করা যায় না।

Exercise 41: Traits — Wrapping Up | এক্সারসাইজ 41: ট্রেইট — সমাপ্তি

📁 **Location:** *exercises/04_traits/14_outro*

📝 **Task (English):**

A final consolidation exercise for the Traits chapter. You will receive minimal guidance and must apply all the trait concepts covered so far: defining and implementing traits, derive macros, operator overloading, trait bounds, `From`/`Into`, `Clone`/`Copy`/`Drop`, and the differences between associated types and generic parameters. The exercise tests your ability to reason about which trait to use in a given situation and to write clean, idiomatic Rust code that leverages the full power of the trait system.

📝 **কাজ (বাংলা):**

Traits অধ্যায়ের চূড়ান্ত consolidation exercise। ন্যূনতম guidance সহ এতক্ষণ কভার করা সমস্ত trait ধারণা প্রয়োগ করো: trait সংজ্ঞায়িত ও ইমপ্লিমেন্ট করা, derive macro, operator overloading, trait bound, `From`/`Into`, `Clone`/`Copy`/`Drop`।

📂 Section 5: Modelling a Ticket — Part 2 (Chapter 5)

অধ্যায় ৫: টিকেট মডেলিং — ভাগ ২

Exercise 42: Ticket v2 Introduction | এক্সারসাইজ 42: টিকেট v2 ভূমিকা

📁 **Location:** *exercises/05_ticket_v2/00_intro*

📝 **Task (English):**

Introduction to the second phase of ticket modelling. This chapter covers: enums for data modelling, the `match` expression, `Option<T>` for nullable values, `Result<T, E>` for fallible operations, error handling patterns, the `Error` trait, Rust's package/crate system, external dependencies with Cargo, the `thiserror` crate, `TryFrom`/`TryInto` traits, and error chaining with `Error::source`. By the end you will have a much more robust, type-safe ticket implementation.

📝 **কাজ (বাংলা):**

টিকেট মডেলিং-এর দ্বিতীয় পর্যায়ের ভূমিকা। এই অধ্যায়ে কভার: data modelling-এর জন্য enum, `match` expression, nullable মানের জন্য `Option<T>`, fallible operations-এর জন্য `Result<T, E>`, error handling pattern, `Error` trait, `thiserror` crate, `TryFrom`/`TryInto`।

Exercise 43: Enumerations | এক্সারসাইজ 43: এনুমেরেশন

📁 **Location:** *exercises/05_ticket_v2/01_enum*

📝 **Task (English):**

Define an enum to model the `Status` field of a ticket. An `enum` (enumeration) defines a type that can be one of several named variants. Replace the `String` status field with a proper `Status` enum that has variants `ToDo`, `InProgress`, and `Done`. Using an enum instead of a raw string prevents invalid states (you can no longer accidentally set the status to "Funny"). Update `Ticket::new` and other related code to work with `Status` values. Remember that `enum` creates a new type, just like `struct`.

📝 **কাজ (বাংলা):**

Ticket-এর `Status` field মডেল করতে enum সংজ্ঞায়িত করো। `enum` একটি টাইপ সংজ্ঞায়িত করে যা বেশ কয়েকটি named variant-এর একটি হতে পারে। `ToDo`, `InProgress`, এবং `Done` variant সহ একটি `Status` enum তৈরি করো।

Exercise 44: Pattern Matching with match | এক্সারসাইজ 44: match দিয়ে প্যাটার্ন ম্যাচিং

📁 **Location:** *exercises/05_ticket_v2/02_match*

📝 **Task (English):**

Use the `match` expression to exhaustively handle all variants of an enum. `match` is Rust's most powerful control-flow construct. It compares a value against a series of patterns and executes the code for the matching arm. The compiler enforces exhaustiveness: you must handle every possible variant or include a catch-all `_` arm. Practice using `match` on your `Status` enum to produce different output for each variant. `match` can also destructure enum variants that carry data, bind names to the extracted values, and use guard expressions.

📝 **কাজ (বাংলা):**

`match` expression ব্যবহার করে enum-এর সব variant exhaustively সামলাও। `match` হলো Rust-এর সবচেয়ে শক্তিশালী control-flow construct। Compiler exhaustiveness enforce করে। তোমার `Status` enum-এ `match` ব্যবহার করো।

Exercise 45: Enum Variants with Data | এক্সারসাইজ 45: ডেটা সহ Enum Variant

📁 **Location:** *exercises/05_ticket_v2/03_variants_with_data*

📝 **Task (English):**

Extend your enum so that variants can carry data. Unlike simple C-style enums, Rust enum variants can hold fields — they can be tuple-like (unnamed fields) or struct-like (named fields). For example, `Status::InProgress { assigned_to: String }` carries the name of the person the ticket is assigned to. Update the `Status` enum and all related `match` expressions to handle the new data-carrying variant. Use destructuring patterns in `match` arms to extract the data from inside a variant.

📝 **কাজ (বাংলা):**

Variant-এ ডেটা বহন করতে পারে এমনভাবে enum extend করো। Rust enum variant-এ field থাকতে পারে — tuple-like (unnamed field) বা struct-like (named field)। উদাহরণ: `Status::InProgress { assigned_to: String }`।

Exercise 46: Concise Branching — if let / let else | এক্সারসাইজ 46: সংক্ষিপ্ত ব্রাঞ্চিং — if let / let else

📁 **Location:** *exercises/05_ticket_v2/04_if_let*

📝 **Task (English):**

Use `if let` and `let`/`else` for concise single-variant pattern matching. When you only care about one specific variant of an enum and want to ignore the rest, `if let` is more readable than a full `match`. Syntax: `if let Pattern = expression { body }`. The `let`/`else` construct is similar but designed for the case where you want to extract a value from a pattern or terminate early if it doesn't match: `let Pattern = expression else { return; }`. Both are idiomatic Rust for reducing boilerplate when full exhaustiveness is not needed.

📝 **কাজ (বাংলা):**

সংক্ষিপ্ত single-variant pattern matching-এর জন্য `if let` এবং `let`/`else` ব্যবহার করো। যখন শুধুমাত্র একটি নির্দিষ্ট variant সম্পর্কে আগ্রহ আছে, `if let` পূর্ণ `match`-এর চেয়ে বেশি পাঠযোগ্য।

Exercise 47: Nullability — Option<T> | এক্সারসাইজ 47: নালযোগ্যতা — Option<T>

📁 **Location:** *exercises/05_ticket_v2/05_nullability*

📝 **Task (English):**

Use `Option<T>` to represent values that may or may not be present. Rust has no null pointer — instead, the standard library provides `Option<T>` with two variants: `Some(T)` (value is present) and `None` (value is absent). This forces callers to explicitly handle the absent case, preventing null-pointer exceptions. Practice wrapping a nullable field (e.g. `assigned_to: Option<String>`) in your `Ticket` struct. Use `match`, `if let`, `.is_some()`, `.is_none()`, `.unwrap()`, `.unwrap_or()`, and `.map()` to work with `Option` values.

📝 **কাজ (বাংলা):**

উপস্থিত বা অনুপস্থিত থাকতে পারে এমন মান উপস্থাপন করতে `Option<T>` ব্যবহার করো। Rust-এ কোনো null pointer নেই; পরিবর্তে `Option<T>` দুটি variant সহ: `Some(T)` এবং `None`। `Ticket` struct-এ `assigned_to: Option<String>` field যোগ করো।

Exercise 48: Fallibility — Result<T, E> | এক্সারসাইজ 48: ব্যর্থতা — Result<T, E>

📁 **Location:** *exercises/05_ticket_v2/06_fallibility*

📝 **Task (English):**

Use `Result<T, E>` for operations that can fail. `Result` has two variants: `Ok(T)` for success and `Err(E)` for failure. Using `Result` makes fallibility explicit in function signatures, forcing callers to handle errors. Modify `Ticket::new` to return `Result<Ticket, ValidationError>` instead of panicking on invalid input. Define a `ValidationError` enum (or struct) to represent the possible validation failures. This is the idiomatic Rust approach to error handling for expected, recoverable errors.

📝 **কাজ (বাংলা):**

ব্যর্থ হতে পারে এমন operations-এর জন্য `Result<T, E>` ব্যবহার করো। `Result`-এর দুটি variant: সফলতার জন্য `Ok(T)` এবং ব্যর্থতার জন্য `Err(E)`। invalid input-এ panic করার পরিবর্তে `Ticket::new` পরিবর্তন করো যাতে `Result<Ticket, ValidationError>` রিটার্ন করে।

Exercise 49: Unwrapping | এক্সারসাইজ 49: আনর্যাপিং

📁 **Location:** *exercises/05_ticket_v2/07_unwrap*

📝 **Task (English):**

Learn about methods to extract the value from `Option` and `Result`. `.unwrap()` extracts the inner value if it is `Some` or `Ok`, and panics otherwise. `.expect("message")` is similar but includes a custom panic message. Use `unwrap` and `expect` sparingly — in production code, prefer explicit error handling via `match` or the `?` operator. They are acceptable in tests, examples, and prototypes. Practice using `match` as an alternative: `match parse_int("42") { Ok(n) => n, Err(e) => panic!("{}", e) }`.

📝 **কাজ (বাংলা):**

`Option` এবং `Result` থেকে মান extract করার মেথড সম্পর্কে জানো। `.unwrap()` `Some` বা `Ok` হলে inner মান extract করে, অন্যথায় panic করে। `.expect("message")` একই কিন্তু custom panic message সহ। `unwrap` সংযতভাবে ব্যবহার করো।

Exercise 50: Error Enums | এক্সারসাইজ 50: এরর এনুম

📁 **Location:** *exercises/05_ticket_v2/08_error_enums*

📝 **Task (English):**

Model errors using enums for precise error handling. Instead of using a single string for all errors, define an enum where each variant represents a distinct error case. For example: `enum TicketError { TitleTooLong, TitleEmpty, DescriptionTooLong, DescriptionEmpty, InvalidStatus }`. Using an error enum allows callers to match on specific error variants and react accordingly, rather than parsing error messages. Practice writing a `match` expression that handles different error variants with different logic: e.g. returning a default value, retrying, or logging.

📝 কাজ (বাংলা):

সুনির্দিষ্ট error handling-এর জন্য enum ব্যবহার করে error মডেল করো। সব error-এর জন্য একক string-এর পরিবর্তে, প্রতিটি variant একটি ভিন্ন error case উপস্থাপন করে এমন enum সংজ্ঞায়িত করো। Error variant match করার অনুশীলন করো।

Exercise 51: The Error Trait | এক্সারসাইজ 51: Error ট্রেইট

📁 **Location:** *exercises/05_ticket_v2/09_error_trait*

📝 **Task (English):**

Implement the `std::error::Error` trait for custom error types. The `Error` trait requires `Display` and `Debug` to be implemented (as super-traits). `Display` provides a human-readable message for end users; `Debug` provides a developer-oriented representation. Implement `Display` for your error enum using the `fmt::Display` trait: `impl fmt::Display for MyError { fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result { ... } }`. The `Error` trait also has an optional `source()` method for error chaining, which we'll explore in a later exercise.

📝 কাজ (বাংলা):

Custom error type-এর জন্য `std::error::Error` trait ইমপ্লিমেন্ট করো। `Error` trait-এর জন্য `Display` এবং `Debug` ইমপ্লিমেন্ট করা প্রয়োজন। `Display` শেষ ব্যবহারকারীদের জন্য human-readable message প্রদান করে।

Exercise 52: Libraries and Binaries | এক্সারসাইজ 52: লাইব্রেরি এবং বাইনারি

📁 **Location:** *exercises/05_ticket_v2/10_packages*

📝 **Task (English):**

Understand the difference between library crates and binary crates. A library crate (`src/lib.rs`) exposes types and functions for use by other crates. A binary crate (`src/main.rs`) provides an entry point with a `main` function and produces an executable. A Cargo package can contain both. Practice splitting your ticket code into a library crate and a small binary that uses it. Understand the `use` keyword for bringing items from the library into scope in the binary, and how to reference the crate by its name in `Cargo.toml`.

📝 কাজ (বাংলা):

library crate এবং binary crate-এর মধ্যে পার্থক্য বোঝো। Library crate (`src/lib.rs`) অন্য crate-এর জন্য type ও function expose করে। Binary crate (`src/main.rs`) `main` ফাংশন সহ executable তৈরি করে।

Exercise 53: External Dependencies | এক্সারসাইজ 53: বাহ্যিক নির্ভরতা

📁 **Location:** *exercises/05_ticket_v2/11_dependencies*

📝 **Task (English):**

Add and use external crate dependencies with Cargo. Open `Cargo.toml` and add a dependency under `[dependencies]`, for example `anyhow = "1"`. Run `cargo build` to download and compile the dependency. Then use the crate in your code with `use anyhow::...`. Understand semantic versioning (SemVer) and how Cargo resolves compatible versions. The course uses `thiserror` in

the next exercise, so this is a warm-up for working with external crates. Also learn about dev-dependencies (`[dev-dependencies]`) which are only compiled for tests.

📝 কাজ (বাংলা):

Cargo দিয়ে বাহ্যিক crate dependency যোগ ও ব্যবহার করো। `Cargo.toml`-এ `[dependencies]`-এর অধীনে dependency যোগ করো। `cargo build` চালাও। Semantic versioning (SemVer) এবং Cargo কীভাবে compatible version resolve করে তা বোঝো।

Exercise 54: The thiserror Crate | এক্সারসাইজ 54: thiserror Crate

📁 **Location:** *exercises/05_ticket_v2/12_thiserror*

📝 **Task (English):**

Use the `thiserror` crate to derive the `Error` trait implementation ergonomically. `thiserror` provides a derive macro (`#[derive(thiserror::Error)]`) that automatically generates `Display` and `Error` implementations from annotations. Use `#[error("message")]` on each variant to specify the display message; use `{0}` or `{field}` in the message to include variant data. This eliminates boilerplate while keeping full control over error messages. Convert your hand-written `Error` implementation from the previous exercise to use `thiserror`.

📝 কাজ (বাংলা):

`thiserror` crate ব্যবহার করে ergonomically `Error` trait implementation derive করো। `#[derive(thiserror::Error)]` macro স্বয়ংক্রিয়ভাবে `Display` এবং `Error` implementation generate করে। প্রতিটি variant-এ `#[error("message")]` ব্যবহার করো।

Exercise 55: TryFrom and TryInto | এক্সারসাইজ 55: TryFrom এবং TryInto

📁 **Location:** *exercises/05_ticket_v2/13_try_from*

📝 **Task (English):**

Implement fallible conversions using the `TryFrom` and `TryInto` traits. Unlike `From`/`Into` (which are infallible), `TryFrom` returns a `Result` and allows the conversion to fail with an error. Implement `TryFrom<String>` for `Status` so that `"To-Do".try_into()` returns `Ok(Status::ToDo)` and an unrecognised string returns `Err(...)`. Just like `From`/`Into`, `TryFrom` and `TryInto` are dual traits — implementing one automatically provides the other. Use `TryInto` in `Ticket::new` to validate and convert the status string.

📝 কাজ (বাংলা):

`TryFrom` এবং `TryInto` trait ব্যবহার করে fallible conversion ইমপ্লিমেন্ট করো। `From`/`Into` (infallible) থেকে ভিন্ন, `TryFrom` একটি `Result` রিটার্ন করে। `Status`-এর জন্য `TryFrom<String>` ইমপ্লিমেন্ট করো।

Exercise 56: Error::source — Error Chaining | এক্সারসাইজ 56: Error::source — Error চেইনিং

📁 **Location:** *exercises/05_ticket_v2/14_source*

📝 **Task (English):**

Implement error chaining using the `Error::source()` method. When one error wraps another (e.g. a `TicketError::ParseFailed` wrapping a `std::num::ParseIntError`), `source()` returns a reference to the underlying error. This forms an error chain that tools and logging frameworks can traverse to show the root cause. With `thiserror`, use the `#[source]` attribute or `#[from]` (which also implements `From`) to annotate the wrapped error field. Practice using the `?` operator to propagate errors automatically — it converts the error type using `From` if needed.

📝 কাজ (বাংলা):

`Error::source()` মেথড ব্যবহার করে error chaining ইমপ্লিমেন্ট করো। যখন একটি error অন্যটি wrap করে, `source()` underlying error-এর reference রিটার্ন করে। `thiserror`-এ `#[source]` বা `#[from]` অ্যাট্রিবিউট ব্যবহার করো। `?` operator ব্যবহার করে error propagate করো।

Exercise 57: Ticket v2 — Wrapping Up | এক্সারসাইজ 57: টিকেট v2 — সমাপ্তি

📁 **Location:** *exercises/05_ticket_v2/15_outro*

📝 **Task (English):**

Final exercise for the Ticket v2 chapter. Bring together enums, `match`, `Option`, `Result`, error types, `TryFrom`/`TryInto`, and error chaining to build a complete, production-quality `Ticket` implementation. The goal is to create a type-safe ticket with proper error handling: all invalid states are rejected at the type level, errors carry meaningful information, and callers can match on specific failure cases. This is the "parse, don't validate" philosophy in practice.

📝 কাজ (বাংলা):

Ticket v2 অধ্যায়ের চূড়ান্ত exercise। enum, `match`, `Option`, `Result`, error type, `TryFrom`/`TryInto`, এবং error chaining একত্রিত করে একটি complete, production-quality `Ticket` ইমপ্লিমেন্টেশন তৈরি করো।

📂 Section 6: Ticket Management (Chapter 6)

অধ্যায় ৬: টিকেট ম্যানেজমেন্ট

Exercise 58: Ticket Management Introduction | এক্সারসাইজ 58: টিকেট ম্যানেজমেন্ট ভূমিকা

📁 **Location:** *exercises/06_ticket_management/00_intro*

📝 **Task (English):**

Introduction to the ticket management chapter, which covers collections and iterators. Topics include: stack-allocated arrays, `Vec<T>` (the growable array), the `Iterator` and `IntoIterator` traits, slices (both immutable `&[T]` and mutable `&mut [T]`), the `Index` and `IndexMut` traits, `HashMap<K, V>` and `BTreeMap<K, V>`. You will build a ticket management system that can store, retrieve, update, and delete tickets using these collection types.

📝 কাজ (বাংলা):

টিকেট ম্যানেজমেন্ট অধ্যায়ের ভূমিকা, যা collection এবং iterator কভার করে। Topics: stack-allocated array, `Vec<T>`, `Iterator` এবং `IntoIterator` trait, slice (`&[T]` এবং `&mut [T]`), `Index` ও `IndexMut` trait, `HashMap` এবং `BTreeMap`।

Exercise 59: Arrays | এক্সারসাইজ 59: অ্যারে

📁 **Location:** *exercises/06_ticket_management/01_arrays*

📝 **Task (English):**

Work with Rust's fixed-size, stack-allocated arrays. An array is declared as `[T; N]` where `T` is the element type and `N` is the number of elements (a compile-time constant). The size is part of the type — `[u32; 5]` and `[u32; 6]` are different types. Array elements are accessed with `array[index]` (zero-indexed). The size of an array in bytes is `size_of::<T>() * N`. Practice creating arrays,

accessing elements, and iterating over them. Also explore multi-dimensional arrays (arrays of arrays).

📝 কাজ (বাংলা):

Rust-এর fixed-size, stack-allocated array নিয়ে কাজ করো। Array `[T; N]` হিসেবে ঘোষণা করা হয়, যেখানে `T` element টাইপ এবং `N` element-এর সংখ্যা (compile-time constant)। Array element `array[index]` দিয়ে access করা হয়।

Exercise 60: Vectors | এক্সারসাইজ 60: ভেক্টর

📁 **Location:** *exercises/06_ticket_management/02_vec*

📝 **Task (English):**

Use `Vec<T>`, Rust's growable, heap-allocated array type. A `Vec` stores its elements contiguously in heap memory and can grow dynamically. Create a `Vec` with `Vec::new()`, `vec![1, 2, 3]`, or `Vec::with_capacity(n)`. Add elements with `.push(value)`, remove the last element with `.pop()`, and access elements with `vec[index]`. Note that `String` is implemented as a `Vec<u8>` internally. Understand the `Vec`'s memory layout: a pointer to the heap buffer, a length (number of elements), and a capacity (allocated buffer size).

📝 কাজ (বাংলা):

Rust-এর growable, heap-allocated array টাইপ `Vec<T>` ব্যবহার করো। `Vec::new()`, `vec![1, 2, 3]`, বা `Vec::with_capacity(n)` দিয়ে তৈরি করো। `.push(value)` দিয়ে element যোগ করো, `.pop()` দিয়ে শেষ element সরাও।

Exercise 61: Resizing a Vec | এক্সারসাইজ 61: Vec রিসাইজ করা

📁 **Location:** *exercises/06_ticket_management/03_resizing*

📝 **Task (English):**

Understand how `Vec` grows and shrinks. When you push an element beyond the current capacity, `Vec` allocates a new, larger buffer (typically doubling in size), copies all existing elements, and frees the old buffer. To avoid repeated reallocations, pre-allocate with `Vec::with_capacity(n)` if you know the approximate size in advance. Use `.reserve(n)` to ensure at least `n` additional slots. `.shrink_to_fit()` releases unused capacity. Implement a function that efficiently builds a `Vec<Ticket>` from a collection of inputs without unnecessary reallocations.

📝 কাজ (বাংলা):

`Vec` কীভাবে বাড়ে ও ছোট হয় তা বোঝো। capacity-এর বাইরে push করলে `Vec` একটি নতুন, বড় buffer allocate করে। অপ্রয়োজনীয় reallocation এড়াতে `Vec::with_capacity(n)` ব্যবহার করো।

Exercise 62: Iterators | এক্সারসাইজ 62: ইটারেটর

📁 **Location:** *exercises/06_ticket_management/04_iterators*

📝 **Task (English):**

Use the `Iterator` and `IntoIterator` traits to iterate over collections. `IntoIterator` converts a collection into an iterator. `Iterator` provides the `next()` method and dozens of adapter methods (`map`, `filter`, `fold`, `sum`, etc.). A `for` loop in Rust calls `.into_iter()` on the collection automatically. Practice consuming an iterator with `for`, `.collect()`, `.sum()`, and `.count()`. Understand the difference between consuming an iterator (takes ownership), iterating by reference (`.iter()` → `&T`), and iterating with mutable references (`.iter_mut()` → `&mut T`).

📝 কাজ (বাংলা):

`Iterator` এবং `IntoIterator` trait ব্যবহার করে collection iterate করো। `for` লুপ স্বয়ংক্রিয়ভাবে `.into_iter()` কল করে। `map`, `filter`, `fold`, `sum` ইত্যাদি adapter মেথড অনুশীলন করো।

Exercise 63: The .iter() Method | এক্সারসাইজ 63: .iter() মেথড

📁 **Location:** *exercises/06_ticket_management/05_iter*

📝 **Task (English):**

Distinguish between `.into_iter()`, `.iter()`, and `.iter_mut()`. `.into_iter()` consumes the collection and yields owned values. `.iter()` borrows the collection and yields immutable references (`&T`). `.iter_mut()` borrows the collection mutably and yields mutable references (`&mut T`). When you call `for x in &vec { ... }`, this is syntactic sugar for `for x in vec.iter() { ... }` and `x` is of type `&T`. Use `.iter()` when you want to inspect elements without consuming or modifying the collection.

📝 **কাজ (বাংলা):**

`.into_iter()`, `.iter()`, এবং `.iter_mut()`-এর মধ্যে পার্থক্য করো। `.into_iter()` collection consume করে। `.iter()` immutable reference (`&T`) yield করে। `.iter_mut()` mutable reference (`&mut T`) yield করে।

Exercise 64: Lifetimes | এক্সারসাইজ 64: লাইফটাইম

📁 **Location:** *exercises/06_ticket_management/06_lifetimes*

📝 **Task (English):**

Understand Rust's lifetime annotations for managing reference validity. A lifetime annotation (e.g. `'a`) names a scope during which a reference is valid. The compiler uses lifetime analysis to ensure that references never outlive the data they point to (no dangling pointers). In many cases, lifetimes can be elided (omitted) and inferred by the compiler. Practice adding explicit lifetime annotations to a function that returns a reference to one of its input references. Understand the lifetime elision rules that allow most simple cases to compile without explicit annotations.

📝 **কাজ (বাংলা):**

Reference validity পরিচালনার জন্য Rust-এর lifetime annotation বোঝো। Lifetime annotation (যেমন `'a`) একটি scope নামকরণ করে যার মধ্যে reference valid। Compiler lifetime বিশ্লেষণ ব্যবহার করে নিশ্চিত করে যে reference কখনো তার data-কে outlive করে না।

Exercise 65: Iterator Combinators | এক্সারসাইজ 65: ইটারেটর কম্বিনেটর

📁 **Location:** *exercises/06_ticket_management/07_combinators*

📝 **Task (English):**

Use iterator combinators to write concise data-transformation pipelines. `Iterator` provides many powerful methods: `.map(f)` transforms each element, `.filter(predicate)` keeps only elements that satisfy the predicate, `.flat_map(f)` maps and flattens, `.take(n)` takes the first n elements, `.skip(n)` skips the first n, `.enumerate()` pairs each element with its index, `.zip(other)` pairs elements from two iterators, `.chain(other)` concatenates iterators, `.fold(init, f)` reduces to a single value, `.collect::<Vec<_>>()` collects into a collection. Practice composing these to filter and transform a list of tickets.

📝 **কাজ (বাংলা):**

সংক্ষিপ্ত data-transformation pipeline লিখতে iterator combinator ব্যবহার করো। `.map(f)`, `.filter(predicate)`, `.flat_map(f)`, `.take(n)`, `.skip(n)`, `.enumerate()`, `.zip(other)`, `.fold(init, f)`, `.collect()` অনুশীলন করো।

Exercise 66: impl Trait in Return Position | এক্সারসাইজ 66: Return Position-এ impl Trait

📁 **Location:** *exercises/06_ticket_management/08_impl_trait*

📝 **Task (English):**

Use `impl Trait` in the return position of a function to hide the concrete return type. `fn get_titles(tickets: &[Ticket]) -> impl Iterator<Item = &str>` tells the caller that the function returns something that implements `Iterator<Item = &str>` without committing to the exact type. This is useful for returning iterators whose concrete type names are complex or private. The `impl Trait` in return position (RPIT) feature is sometimes called an opaque return type. Note that the concrete type is fixed per call site — the function must always return the same type.

📝 কাজ (বাংলা):

Concrete return type লুকাতে ফাংশনের return position-এ `impl Trait` ব্যবহার করো। `fn get_titles(tickets: &[Ticket]) -> impl Iterator<Item = &str>` caller-কে জানায় যে ফাংশন `Iterator<Item = &str>` ইমপ্লিমেন্ট করে এমন কিছু রিটার্ন করে।

Exercise 67: impl Trait in Argument Position | এক্সারসাইজ 67: Argument Position-এ impl Trait

📁 **Location:** *exercises/06_ticket_management/09_impl_trait_2*

📝 **Task (English):**

Use `impl Trait` in function argument position as a shorthand for a generic with a trait bound. `fn print(item: impl Display)` is equivalent to `fn print<T: Display>(item: T)`. Both are monomorphised at compile time (no dynamic dispatch). However, with `impl Trait` in argument position, the caller cannot explicitly specify the type (there is no `print::<String>(...)`). As a rule of thumb, prefer generic type parameters (`<T: Trait>`) over `impl Trait` in argument position, because generics allow the caller to name the type explicitly when needed.

📝 কাজ (বাংলা):

Trait bound সহ generic-এর shorthand হিসেবে ফাংশন argument position-এ `impl Trait` ব্যবহার করো। `fn print(item: impl Display)` এবং `fn print<T: Display>(item: T)` সমতুল্য। নিয়ম: argument position-এ `impl Trait`-এর চেয়ে generic type parameter পছন্দনীয়।

Exercise 68: Slices | এক্সারসাইজ 68: স্লাইস

📁 **Location:** *exercises/06_ticket_management/10_slices*

📝 **Task (English):**

Work with slices — views into a contiguous sequence of elements in memory. A slice `&[T]` is a fat pointer: a pointer to the start of the sequence plus the length. You can create a slice from an array or a `Vec` using range syntax: `&array[1..3]`. Functions that accept `&[T]` can work with both arrays and `Vec`s (the `Deref` trait provides the coercion). Understand that slices do not own their data — they borrow it. Practice writing functions that accept `&[Ticket]` and return information about the collection without taking ownership.

📝 কাজ (বাংলা):

মেমরিতে contiguous sequence-এর view — slice নিয়ে কাজ করো। একটি slice `&[T]` হলো একটি fat pointer: sequence-এর শুরুর pointer এবং length। Array বা `Vec` থেকে range syntax: `&array[1..3]` ব্যবহার করে slice তৈরি করো।

Exercise 69: Mutable Slices | এক্সারসাইজ 69: মিউটেবল স্লাইস

📁 **Location:** *exercises/06_ticket_management/11_mutable_slices*

📝 **Task (English):**

Use mutable slices `&mut [T]` to modify elements in place. A mutable slice allows you to read and write the elements of the underlying collection. You cannot call `Vec`-specific methods (like `.push()`) on a slice, but you can modify existing elements. Practice sorting a slice with `.sort()` or `.sort_by(|a, b| ...)`, reversing with `.reverse()`, and modifying individual elements via index. Understand Rust's rule: you can have either one mutable reference or many immutable references to data, but not both simultaneously.

📝 **কাজ (বাংলা):**

Elements in-place পরিবর্তন করতে mutable slice `&mut [T]` ব্যবহার করো। Mutable slice underlying collection-এর element পড়তে ও লিখতে দেয়। `.sort()`, `.sort_by(...)`, `.reverse()` অনুশীলন করো।

Exercise 70: Type-State Pattern (Two States) | এক্সারসাইজ 70: টাইপ-স্টেট প্যাটার্ন (দুই অবস্থা)

📁 **Location:** *exercises/06_ticket_management/12_two_states*

📝 **Task (English):**

Apply the type-state pattern to model domain invariants at the type level. Instead of a single `Ticket` type with an optional `id` field, create two separate types: `TicketDraft` (no id, no status — a ticket not yet saved) and `Ticket` (has an id and status — a saved ticket). This prevents invalid operations at compile time: you cannot accidentally read the id of a draft. Practice converting a `TicketDraft` into a `Ticket` using `From<TicketDraft>` or a method, and understand how the type system becomes part of your domain model.

📝 **কাজ (বাংলা):**

টাইপ level-এ domain invariant মডেল করতে type-state pattern প্রয়োগ করো। একটি optional `id` field সহ একক `Ticket` type-এর পরিবর্তে দুটি পৃথক type তৈরি করো: `TicketDraft` (id নেই) এবং `Ticket` (id আছে)।

Exercise 71: The Index Trait | এক্সারসাইজ 71: Index ট্রেইট

📁 **Location:** *exercises/06_ticket_management/13_index*

📝 **Task (English):**

Implement the `Index` trait to allow bracket indexing (`store[id]`) on your ticket store. `Index` requires implementing `fn index(&self, index: Idx) -> &Self::Output`. Note that `index` does not return `Option` — by convention, it panics on out-of-bounds access. Define a `TicketStore` struct wrapping a `Vec<Ticket>` and a unique-ID counter, then implement `Index<TicketId>` so that `store[id]` returns a reference to the matching ticket. Panicking on an invalid id is acceptable for this use case since it mirrors how `Vec` indexing works.

📝 **কাজ (বাংলা):**

Ticket store-এ bracket indexing (`store[id]`) সক্ষম করতে `Index` trait ইমপ্লিমেন্ট করো। `Index` trait-এ `fn index(&self, index: Idx) -> &Self::Output` ইমপ্লিমেন্ট করতে হয়। একটি `TicketStore` struct সংজ্ঞায়িত করো এবং `Index<TicketId>` ইমপ্লিমেন্ট করো।

Exercise 72: The IndexMut Trait | এক্সারসাইজ 72: IndexMut ট্রেইট

📁 **Location:** *exercises/06_ticket_management/14_index_mut*

📝 **Task (English):**

Implement the `IndexMut` trait to allow mutable indexing (`store[id] = new_ticket`) on your ticket store. `IndexMut` builds on `Index` (a type must implement `Index` before it can implement `IndexMut`) and requires `fn index_mut(&mut self, index: Idx) -> &mut Self::Output`. This enables patterns like `store[id].title = "New Title".into()`. Implement `IndexMut<TicketId>` for `TicketStore` and verify that you can modify a ticket's fields via mutable indexing.

📝 কাজ (বাংলা):

Ticket store-এ mutable indexing (`store[id] = new_ticket`) সক্ষম করতে `IndexMut` trait ইমপ্লিমেন্ট করো। `IndexMut` `Index`-এর উপর নির্মিত এবং `fn index_mut(&mut self, index: Idx) -> &mut Self::Output` প্রয়োজন।

Exercise 73: HashMap | এক্সারসাইজ 73: হ্যাশম্যাপ

📁 **Location:** *exercises/06_ticket_management/15_hashmap*

📝 **Task (English):**

Use `HashMap<K, V>` to store tickets by their id for O(1) average-case lookups. `HashMap` requires keys to implement `Eq` and `Hash`. Insert with `.insert(key, value)`, retrieve with `.get(&key)` (returns `Option<&V>`), remove with `.remove(&key)`, and check existence with `.contains_key(&key)`. Understand the implicit contract: if two keys are equal (`Eq`), their hashes must be equal too. Derive `Hash` and `Eq` for your `TicketId` type and refactor `TicketStore` to use a `HashMap` instead of a `Vec`.

📝 কাজ (বাংলা):

O(1) average-case lookup-এর জন্য id দিয়ে ticket সংরক্ষণ করতে `HashMap<K, V>` ব্যবহার করো। `HashMap`-এ key-এর জন্য `Eq` এবং `Hash` ইমপ্লিমেন্ট করতে হয়। `TicketStore`-কে `Vec`-এর পরিবর্তে `HashMap` ব্যবহার করতে refactor করো।

Exercise 74: BTreeMap and Ordering | এক্সারসাইজ 74: BTreeMap এবং ক্রম

📁 **Location:** *exercises/06_ticket_management/16_btreemap*

📝 **Task (English):**

Use `BTreeMap<K, V>` when you need a sorted map. Unlike `HashMap`, `BTreeMap` keeps keys in sorted order and requires keys to implement `Ord`. Iteration over a `BTreeMap` visits entries in key order. The trade-off: lookups are O(log n) vs. O(1) for `HashMap`, but you get sorted iteration and range queries for free. Implement `Ord` and `PartialOrd` for your key type (ensuring consistency with `Eq`). Practice using range queries on `BTreeMap` with `.range(start..end)` to retrieve tickets within a specific ID range.

📝 কাজ (বাংলা):

Sorted map দরকার হলে `BTreeMap<K, V>` ব্যবহার করো। `HashMap`-এর বিপরীতে `BTreeMap` key sorted order-এ রাখে এবং key-এর জন্য `Ord` প্রয়োজন। Lookup O(log n) কিন্তু sorted iteration পাওয়া যায়।

📂 Section 7: Threads (Chapter 7)

অধ্যায় ৭: থ্রেড

Exercise 75: Threads Introduction | এক্সারসাইজ 75: থ্রেড ভূমিকা

📁 **Location:** *exercises/07_threads/00_intro*

📝 **Task (English):**

Introduction to concurrency with Rust threads. This chapter covers: spawning threads, the `'static` lifetime bound for thread data, leaking data for threads, scoped threads, message-passing channels

(`std::sync::mpsc`), interior mutability, `Mutex<T>`, `Arc<T>`, `RwLock<T>`, the `Send` and `Sync` marker traits, and various concurrency design patterns for a multi-threaded ticket server. By the end you will have a functioning multi-threaded ticket system.

📝 কাজ (বাংলা):

Rust thread-এর সাথে concurrency-র ভূমিকা। Topics: thread spawn করা, `'static` lifetime bound, scoped thread, message-passing channel, interior mutability, `Mutex<T>`, `Arc<T>`, `RwLock<T>`, `Send` এবং `Sync` marker trait।

Exercise 76: Spawning Threads | এক্সারসাইজ 76: থ্রেড স্পন করা

📁 **Location:** *exercises/07_threads/01_threads*

📝 **Task (English):**

Spawn OS threads using `std::thread::spawn`. The closure passed to `spawn` runs on a new thread. `spawn` returns a `JoinHandle<T>` — call `.join()` to wait for the thread to finish and retrieve its return value (or propagate its panic). Practice spawning multiple threads that each perform a computation and then collecting their results. Understand that the spawned thread must satisfy `'static` — it cannot borrow stack data from the parent thread (because the parent might return first). Use `move` closures to transfer ownership of data into the spawned thread.

📝 কাজ (বাংলা):

`std::thread::spawn` ব্যবহার করে OS thread spawn করো। `spawn`-এ pass করা closure একটি নতুন thread-এ চলে। `spawn` একটি `JoinHandle<T>` রিটার্ন করে — `.join()` কল করো thread শেষ হওয়ার জন্য অপেক্ষা করতে।

Exercise 77: The 'static Lifetime Bound | এক্সারসাইজ 77: 'static লাইফটাইম বাউন্ড

📁 **Location:** *exercises/07_threads/02_static*

📝 **Task (English):**

Understand why spawned threads require `'static` data. `std::thread::spawn` requires its closure to be `'static` — meaning the closure must not contain any references that could become dangling while the thread is running. This is because the spawned thread might outlive the scope in which it was created. Data stored in static variables (with the `static` keyword) lives for the duration of the program and satisfies `'static`. Owned values (`String`, `Vec`, etc.) also satisfy `'static` because they do not borrow from any scope. Practice wrapping shared data in `Arc` to meet this requirement.

📝 কাজ (বাংলা):

কেন spawned thread-এর `'static` data দরকার তা বোঝো। `std::thread::spawn`-এর closure `'static` হতে হবে — closure-এ এমন কোনো reference থাকা চলবে না যা thread চলার সময় dangling হতে পারে। `Arc` ব্যবহার করে shared data wrap করো।

Exercise 78: Leaking Data | এক্সারসাইজ 78: ডেটা লিক করা

📁 **Location:** *exercises/07_threads/03_leak*

📝 **Task (English):**

Use `Box::leak` to create a `'static` reference by deliberately leaking heap memory. `Box::leak(boxed_value)` returns `&'static mut T` — the memory is never freed. This is a valid strategy when you need a `'static` reference and the data truly should live forever (e.g. configuration loaded at startup). Understand the trade-offs: you gain a `'static` reference at the cost of a memory leak. This technique is sometimes used for global configuration. Practice leaking a configuration struct and sharing it across threads.

📝 কাজ (বাংলা):

Heap মেমরি ইচ্ছাকৃতভাবে leak করে `'static` reference তৈরি করতে `Box::leak` ব্যবহার করো। `Box::leak(boxed_value)` `&'static mut T` রিটার্ন করে — মেমরি কখনো মুক্ত হয় না। Configuration struct leak করো এবং thread জুড়ে share করো।

Exercise 79: Scoped Threads | এক্সারসাইজ 79: স্কোপড থ্রেড

📁 **Location:** *exercises/07_threads/04_scoped_threads*

📝 **Task (English):**

Use `std::thread::scope` to spawn threads that can borrow data from the enclosing scope. Unlike `spawn`, scoped threads are guaranteed to finish before the scope ends, so they can safely hold references to stack data. Inside the `scope` closure, use `scope.spawn(...)` to create threads. The scope ensures all spawned threads are joined before `thread::scope` returns. This eliminates the need for `Arc` and the `'static` bound when sharing read-only data. Practice refactoring a `spawn`-based implementation to use scoped threads.

📝 কাজ (বাংলা):

Enclosing scope থেকে data borrow করতে পারে এমন thread spawn করতে `std::thread::scope` ব্যবহার করো। Scoped thread scope শেষ হওয়ার আগে শেষ হওয়ার guarantee দেয়, তাই stack data-র reference নিরাপদে hold করতে পারে।

Exercise 80: Channels | এক্সারসাইজ 80: চ্যানেল

📁 **Location:** *exercises/07_threads/05_channels*

📝 **Task (English):**

Use `std::sync::mpsc` (multi-producer, single-consumer) channels for message passing between threads. Create a channel with `mpsc::channel()` which returns a `(Sender<T>, Receiver<T>)` pair. Send messages with `sender.send(value)` and receive with `receiver.recv()` (blocking). Multiple senders can be created by cloning the `Sender`. Build a simple multi-threaded pipeline where a producer thread sends `Ticket` values through a channel and a consumer thread receives and processes them. Handle the `send` and `recv` errors appropriately.

📝 কাজ (বাংলা):

Thread-এর মধ্যে message passing-এর জন্য `std::sync::mpsc` (multi-producer, single-consumer) channel ব্যবহার করো। `mpsc::channel()` দিয়ে channel তৈরি করো। `sender.send(value)` দিয়ে পাঠাও এবং `receiver.recv()` দিয়ে গ্রহণ করো।

Exercise 81: Interior Mutability | এক্সারসাইজ 81: অভ্যন্তরীণ মিউটেবিলিটি

📁 **Location:** *exercises/07_threads/06_interior_mutability*

📝 **Task (English):**

Understand interior mutability — the ability to mutate data through an immutable reference. `Cell<T>` and `RefCell<T>` provide interior mutability for single-threaded contexts. `RefCell<T>` enforces the borrow rules at runtime instead of compile time: `.borrow()` gives an immutable reference and `.borrow_mut()` gives a mutable one; panics if the rules are violated. For multi-threaded contexts, `Mutex<T>` and `RwLock<T>` are the analogues. Practice using `RefCell` in a single-threaded scenario and then understand why it cannot be used across threads (`RefCell` is not `Sync`).

📝 কাজ (বাংলা):

Interior mutability বোঝো — immutable reference-এর মাধ্যমে data mutate করার ক্ষমতা। `Cell<T>` এবং `RefCell<T>` single-threaded context-এ interior mutability প্রদান করে। `RefCell<T>` borrow rule runtime-এ enforce করে।

Exercise 82: Two-Way Communication (Acknowledgement) | এক্সারসাইজ 82: দ্বিমুখী যোগাযোগ (স্বীকৃতি)

📁 **Location:** *exercises/07_threads/07_ack*

📝 **Task (English):**

Implement two-way communication between a client thread and a server thread using a pair of channels. The client sends a command (e.g. create a ticket) together with a one-shot response channel. The server processes the command and sends the result back on the response channel. The client then receives the response. This pattern allows the client to wait for confirmation that the server processed its request. Practice creating a dedicated response `Sender` per request and sending it as part of the command message.

📝 **কাজ (বাংলা):**

Channel-এর pair ব্যবহার করে client thread এবং server thread-এর মধ্যে two-way communication ইমপ্লিমেন্ট করো। Client একটি command এবং একটি one-shot response channel পাঠায়। Server command process করে result পাঠিয়ে দেয়।

Exercise 83: A Dedicated Client Type | এক্সারসাইজ 83: ডেডিকেটেড ক্লায়েন্ট টাইপ

📁 **Location:** *exercises/07_threads/08_client*

📝 **Task (English):**

Encapsulate the client-side channel logic in a dedicated `TicketStoreClient` struct. The struct holds the `Sender` end of the command channel and provides ergonomic methods (`create`, `get`, `update`, `delete`) that handle creating response channels, sending commands, and waiting for responses internally. This hides the low-level channel mechanics from callers and provides a clean API. Practice implementing `Clone` for the client so that multiple threads can each hold their own sender handle.

📝 **কাজ (বাংলা):**

Dedicated `TicketStoreClient` struct-এ client-side channel logic encapsulate করো। Struct command channel-এর `Sender` hold করে এবং ergonomic মেথড (`create`, `get`, `update`, `delete`) প্রদান করে। `Clone` ইমপ্লিমেন্ট করো যাতে একাধিক thread প্রত্যেকে নিজের sender handle ধরে রাখতে পারে।

Exercise 84: Bounded vs. Unbounded Channels | এক্সারসাইজ 84: বাউন্ডেড বনাম আনবাউন্ডেড চ্যানেল

📁 **Location:** *exercises/07_threads/09_bounded*

📝 **Task (English):**

Understand the difference between unbounded and bounded channels and when to use each. `mpsc::channel()` creates an unbounded channel — the sender never blocks, and messages queue up indefinitely (risking unbounded memory growth). `mpsc::sync_channel(n)` creates a bounded channel with capacity `n` — once the buffer is full, `send` blocks until space is available. Bounded channels provide backpressure: they slow down producers when the consumer cannot keep up. Refactor the ticket server to use a bounded channel and observe how it prevents the server from being overwhelmed.

📝 **কাজ (বাংলা):**

Unbounded এবং bounded channel-এর মধ্যে পার্থক্য বোঝো। `mpsc::channel()` unbounded channel তৈরি করে। `mpsc::sync_channel(n)` capacity `n` সহ bounded channel তৈরি করে — buffer পূর্ণ হলে `send` block করে। Bounded channel backpressure প্রদান করে।

Exercise 85: Patch Updates | এক্সারসাইজ 85: প্যাচ আপডেট

📁 **Location:** *exercises/07_threads/10_patch*

📝 **Task (English):**

Implement partial (patch) updates for tickets using the builder/patch pattern. Instead of requiring all fields to be provided for an update, define a `TicketPatch` struct where each field is `Option<T>`. A `None` field means 'do not change this field'; a `Some(value)` means 'update this field to value'. Implement an `update` method on `Ticket` that applies a `TicketPatch`, updating only the fields that are `Some`. This pattern avoids sending large payloads when only a few fields need changing and is a common REST API convention.

📝 **কাজ (বাংলা):**

Builder/patch pattern ব্যবহার করে ticket-এর partial (patch) update ইমপ্লিমেন্ট করো। `TicketPatch` struct সংজ্ঞায়িত করো যেখানে প্রতিটি field `Option<T>`। `None` মানে field পরিবর্তন করো না; `Some(value)` মানে field update করো।

Exercise 86: Locks — Mutex | এক্সারসাইজ 86: লক — Mutex

📁 **Location:** *exercises/07_threads/11_locks*

📝 **Task (English):**

Use `Mutex<T>` for safe shared-state concurrency. A `Mutex` wraps a value and ensures only one thread can access it at a time. Call `.lock()` to acquire the mutex guard (blocks until the lock is free); the guard dereferences to `&mut T`. When the guard goes out of scope, the lock is released automatically. Combine `Mutex<T>` with `Arc<T>` to share a mutex across threads: `Arc<Mutex<TicketStore>>`. Practice building a multi-threaded server that uses `Arc<Mutex<TicketStore>>` instead of channels for state management.

📝 **কাজ (বাংলা):**

Safe shared-state concurrency-র জন্য `Mutex<T>` ব্যবহার করো। `Mutex` একটি value wrap করে এবং নিশ্চিত করে শুধুমাত্র একটি thread একসময় এটি access করতে পারে। Thread জুড়ে mutex share করতে `Arc<Mutex<TicketStore>>` ব্যবহার করো।

Exercise 87: Readers and Writers — RwLock | এক্সারসাইজ 87: রিডার এবং রাইটার — RwLock

📁 **Location:** *exercises/07_threads/12_rw_lock*

📝 **Task (English):**

Use `RwLock<T>` to allow concurrent reads with exclusive writes. `RwLock` provides two modes: shared read (`.read()`) — multiple threads can hold a read lock simultaneously — and exclusive write (`.write()`) — only one thread can hold a write lock. This improves throughput in read-heavy workloads compared to `Mutex`. Replace `Arc<Mutex<TicketStore>>` with `Arc<RwLock<TicketStore>>` and verify that multiple readers can work concurrently while writes are exclusive.

📝 **কাজ (বাংলা):**

Concurrent read এবং exclusive write-এর জন্য `RwLock<T>` ব্যবহার করো। `RwLock` দুটি mode প্রদান করে: shared read (`.read()`) — একাধিক thread একসাথে read lock hold করতে পারে — এবং exclusive write (`.write()`)।

Exercise 88: Shared-State Concurrency Without Channels | এক্সারসাইজ 88: চ্যানেল ছাড়া Shared-State Concurrency

📁 **Location:** *exercises/07_threads/13_without_channels*

📝 **Task (English):**

Refactor the ticket server to use shared state (`Arc<RwLock<TicketStore>>`) instead of channels for its core operations. Access the shared store directly from client threads using the lock. Compare this design with the channel-based design: channels enforce a strict producer–consumer model and naturally serialize access; shared-state with locks is more flexible but requires careful use of lock scope to avoid deadlocks. Understand the trade-offs and choose the right approach for your use case.

📝 **কাজ (বাংলা):**

Core operations-এর জন্য channel-এর পরিবর্তে shared state (`Arc<RwLock<TicketStore>>`) ব্যবহার করতে ticket server refactor করো। Channel-based design এবং shared-state lock design-এর মধ্যে trade-off বোঝো।

Exercise 89: The Sync Trait | এক্সারসাইজ 89: Sync ট্রেইট

📁 **Location:** *exercises/07_threads/14_sync*

📝 **Task (English):**

Understand the `Send` and `Sync` marker traits that govern thread safety. `Send` means: it is safe to transfer ownership of this value to another thread. `Sync` means: it is safe to share a reference to this value across threads (i.e. `&T` is `Send`). Most types implement both automatically. Types that do not are: `Rc<T>` (not `Send` or `Sync`), `RefCell<T>` (not `Sync`), raw pointers (neither). The compiler enforces these at compile time. Practice reasoning about which types are `Send`/`Sync` and why certain designs require `Arc` instead of `Rc`.

📝 **কাজ (বাংলা):**

Thread safety পরিচালনা করে `Send` এবং `Sync` marker trait বোঝো। `Send` মানে: অন্য thread-এ ownership transfer করা নিরাপদ। `Sync` মানে: thread জুড়ে reference share করা নিরাপদ। `Rc<T>` (not `Send` বা `Sync`), `RefCell<T>` (not `Sync`) অনুশীলন করো।

📂 Section 8: Async Rust (Chapter 8)

অধ্যায় ৮: Async Rust

Exercise 90: Async Rust Introduction | এক্সারসাইজ 90: Async Rust ভূমিকা

📁 **Location:** *exercises/08_futures/00_intro*

📝 **Task (English):**

Introduction to asynchronous programming in Rust. This chapter covers: the `async`/`.await` keywords, the `Future` trait, Tokio as the async runtime, spawning async tasks, the runtime architecture (single-threaded vs. multi-threaded schedulers), blocking vs. non-blocking operations, async-aware synchronisation primitives, and cancellation. Asynchronous programming allows a

single thread to manage many concurrent I/O-bound operations without blocking, making it ideal for web servers and network clients.

📝 কাজ (বাংলা):

Rust-এ asynchronous programming-এর ভূমিকা। Topics: `async`/`.await` keyword, `Future` trait, Tokio runtime, async task spawn, blocking বনাম non-blocking operations, async-aware synchronisation primitive, cancellation। Async programming একটি single thread-কে অনেক concurrent I/O-bound operation পরিচালনা করতে দেয়।

Exercise 91: Asynchronous Functions | এক্সারসাইজ 91: অ্যাসিঙ্ক্রোনাস ফাংশন

📁 **Location:** *exercises/08_futures/01_async_fn*

📝 **Task (English):**

Write asynchronous functions using the `async fn` syntax. An `async fn` returns a `Future` — a value representing a computation that has not yet completed. The future is lazy: it does nothing until polled by an executor. Use `.await` to suspend the current async function until the awaited future completes. Use `#[tokio::main]` (or `#[tokio::test]`) to run async code from a synchronous entry point. Practice converting a synchronous function to async and using `.await` to chain asynchronous calls.

📝 কাজ (বাংলা):

`async fn` syntax ব্যবহার করে asynchronous ফাংশন লেখো। একটি `async fn` একটি `Future` রিটার্ন করে। Future lazy: executor দ্বারা poll না হওয়া পর্যন্ত কিছু করে না। `.await` ব্যবহার করো awaited future সম্পূর্ণ না হওয়া পর্যন্ত suspend করতে।

Exercise 92: Spawning Async Tasks | এক্সারসাইজ 92: Async Task স্পন করা

📁 **Location:** *exercises/08_futures/02_spawn*

📝 **Task (English):**

Spawn async tasks using `tokio::spawn`. Like `std::thread::spawn` for threads, `tokio::spawn` creates a new async task that runs concurrently with the current task. `tokio::spawn` returns a `JoinHandle<T>`; call `.await` on the handle to wait for the task. Key difference from threads: all tasks run on the same thread pool managed by Tokio, and task switching happens at `.await` points. Panics in spawned tasks are caught and returned as errors from the `JoinHandle`. Practice spawning multiple tasks and collecting their results.

📝 কাজ (বাংলা):

`tokio::spawn` ব্যবহার করে async task spawn করো। `tokio::spawn` একটি নতুন async task তৈরি করে যা বর্তমান task-এর সাথে concurrently চলে। Handle-এ `.await` কল করো task-এর জন্য অপেক্ষা করতে।

Exercise 93: Runtime Architecture | এক্সারসাইজ 93: রানটাইম আর্কিটেকচার

📁 **Location:** *exercises/08_futures/03_runtime*

📝 **Task (English):**

Understand Tokio's runtime architecture. Tokio provides two scheduler flavours: `#[tokio::main]` with `flavor = "current_thread"` (single-threaded) and `flavor = "multi_thread"` (the default, which uses a thread pool). In the multi-threaded scheduler, tasks may be run on different threads, so they must be `Send`. `Rc<T>` and `RefCell<T>` are not `Send` and cannot be used across `.await` points in a multi-thread runtime. Practice identifying why certain async code fails to compile and how to fix it by switching from `Rc` to `Arc` and from `RefCell` to `Mutex`.

📝 কাজ (বাংলা):

Tokio-র runtime architecture বোঝো। Tokio দুটি scheduler প্রদান করে: `flavor = "current_thread"` (single-threaded) এবং `flavor = "multi_thread"` (default thread pool)। Multi-thread scheduler-এ task `Send` হতে হবে।

Exercise 94: The Future Trait | এক্সারসাইজ 94: Future ট্রেইট

📁 **Location:** *exercises/08_futures/04_future*

📝 **Task (English):**

Implement the `Future` trait manually to understand how async/await works under the hood. The `Future` trait has a single method: `fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>`. `Poll::Ready(value)` means the future has completed; `Poll::Pending` means it has not yet and the executor should be woken up (via the `Waker` in `Context`) when it is ready to make progress. Understand that `.await` is syntactic sugar that continuously polls the future until it returns `Poll::Ready`. Practice writing a simple state-machine future by hand.

📝 **কাজ (বাংলা):**

Async/await কীভাবে কাজ করে তা বুঝতে `Future` trait manually ইমপ্লিমেন্ট করো। `Future` trait-এ একটিমাত্র মেথড: `fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>`। `.await` sugar যা future `Poll::Ready` রিটার্ন করা পর্যন্ত continuously poll করে।

Exercise 95: Don't Block the Async Runtime | এক্সারসাইজ 95: Async Runtime Block করো না

📁 **Location:** *exercises/08_futures/05_blocking*

📝 **Task (English):**

Learn why blocking operations are dangerous in an async runtime and how to avoid them. The async runtime uses a small thread pool to run tasks. If a task blocks (e.g. by sleeping with `std::thread::sleep`, doing heavy CPU work, or performing synchronous I/O), it blocks one of the runtime's worker threads, potentially starving other tasks. Use `tokio::time::sleep` instead of `std::thread::sleep`, async I/O instead of synchronous I/O, and `tokio::task::spawn_blocking` for CPU-intensive or inherently blocking code.

📝 **কাজ (বাংলা):**

Blocking operation async runtime-এ কেন বিপজ্জনক এবং কীভাবে এড়ানো যায় তা শেখো। Async runtime task চালাতে একটি ছোট thread pool ব্যবহার করে। `std::thread::sleep`-এর পরিবর্তে `tokio::time::sleep` ব্যবহার করো, CPU-intensive কোডের জন্য `tokio::task::spawn_blocking` ব্যবহার করো।

Exercise 96: Async-Aware Primitives | এক্সারসাইজ 96: Async-সচেতন Primitive

📁 **Location:** *exercises/08_futures/06_async_aware_primitives*

📝 **Task (English):**

Use async-aware synchronisation primitives from `tokio::sync` instead of the standard library equivalents. When a `tokio::sync::Mutex` is locked, the current task yields (suspends) rather than blocking the thread. When a `std::sync::Mutex` is locked, the thread itself blocks. Replace `std::sync::Mutex` with `tokio::sync::Mutex` in async code that holds the lock across `.await` points. The same applies to `tokio::sync::RwLock`, `tokio::sync::Semaphore`, and `tokio::sync::Notify`. Use the correct primitive for each scenario.

📝 **কাজ (বাংলা):**

Standard library-র পরিবর্তে `tokio::sync` থেকে async-aware synchronisation primitive ব্যবহার করো। `tokio::sync::Mutex` lock হলে বর্তমান task yield করে (thread block করে না)। `.await` point জুড়ে lock hold করলে `tokio::sync::Mutex` ব্যবহার করো।

Exercise 97: Cancellation | এক্সারসাইজ 97: বাতিলকরণ (Cancellation)

📁 **Location:** *exercises/08_futures/07_cancellation*

📝 **Task (English):**

Handle async task cancellation. In Tokio, a task is cancelled by dropping its `JoinHandle` (or by aborting it via `.abort()`). When a task is cancelled, the future is dropped at the next `.await` point — code after that point does not run. This means destructors run, but any operation spanning an `.await` may be left half-done. Be careful with `select!` macro races: if you race two futures and the losing future has partially completed work, that work is discarded. Practice writing cancel-safe code and understand when cancellation is and is not safe.

📝 কাজ (বাংলা):

Async task cancellation সামলাও। Tokio-তে task cancel করা হয় `JoinHandle` drop করে বা `.abort()` দিয়ে। Task cancel হলে, future পরবর্তী `.await` point-এ drop হয়। `select!` macro race-এ সতর্ক থাকো। Cancel-safe কোড লেখার অনুশীলন করো।

Exercise 98: Async Rust — Outro | এক্সারসাইজ 98: Async Rust — সমাপ্তি

📁 **Location:** *exercises/08_futures/08_outro*

📝 **Task (English):**

Final consolidation exercise for the entire course. You will apply everything learned — structs, enums, traits, ownership, iterators, threads, and async/await — to build a complete async ticket management server that can handle concurrent clients. The server should accept commands over an async channel, process them, and return responses. This exercise tests end-to-end understanding of Rust's core concepts and idiomatic patterns. By completing it, you will have the confidence to work on real-world Rust projects.

📝 কাজ (বাংলা):

পুরো course-এর জন্য চূড়ান্ত consolidation exercise। struct, enum, trait, ownership, iterator, thread, এবং async/await সব একত্রিত করে একটি complete async ticket management server তৈরি করো যা concurrent client সামলাতে পারে।

📋 দ্রুত রেফারেন্স সারণি (Quick Reference Table)

No.	Exercise Title	File Location	Section
1	Welcome to Rust	*01_intro/00_welcome*	Welcome & Syntax
2	Rust Syntax Basics	*01_intro/01_syntax*	Basic Calculator
3	Calculator Introduction	*02_basic_calculator/00_intro*	Basic Calculator
4	Integer Types	*02_basic_calculator/01_integers*	Basic Calculator
5	Variables	*02_basic_calculator/02_variables*	Basic Calculator
6	Control Flow — if/else	*02_basic_calculator/03_if_else*	Basic Calculator
7	Panics	*02_basic_calculator/04_panics*	Basic Calculator
8	Factorial (Recursion)	*02_basic_calculator/05_factorial*	Basic Calculator

9	Loops — while	*02_basic_calculator/06_while*	Basic Calculator
10	Loops — for	*02_basic_calculator/07_for*	Basic Calculator
11	Integer Overflow	*02_basic_calculator/08_overflow*	Basic Calculator
12	Saturating Arithmetic	*02_basic_calculator/09_saturating*	Ticket v1
13	Type Casting with as	*02_basic_calculator/10_as_casting*	Ticket v1
14	Ticket Introduction	*03_ticket_v1/00_intro*	Ticket v1
15	Structs	*03_ticket_v1/01_struct*	Ticket v1
16	Validation	*03_ticket_v1/02_validation*	Ticket v1
17	Modules	*03_ticket_v1/03_modules*	Ticket v1
18	Visibility	*03_ticket_v1/04_visibility*	Ticket v1
19	Encapsulation	*03_ticket_v1/05_encapsulation*	Ticket v1
20	Ownership	*03_ticket_v1/06_ownership*	Ticket v1
21	Mutable References / Setters	*03_ticket_v1/07_setters*	Ticket v1
22	Stack Memory Layout	*03_ticket_v1/08_stack*	Ticket v1
23	Heap Allocation	*03_ticket_v1/09_heap*	Ticket v1
24	References in Memory	*03_ticket_v1/10_references_in_memory*	Traits
25	Destructors	*03_ticket_v1/11_destructor*	Traits
26	Ticket — Wrapping Up	*03_ticket_v1/12_outro*	Traits
27	Traits — Introduction	*04_traits/00_intro*	Traits
28	Defining and Implementing Traits	*04_traits/01_trait*	Traits
29	The Orphan Rule	*04_traits/02_orphan_rule*	Traits
30	Operator Overloading	*04_traits/03_operator_overloading*	Traits
31	Derive Macros	*04_traits/04_derive*	Traits
32	Trait Bounds	*04_traits/05_trait_bounds*	Traits
33	String Slices (&str)	*04_traits/06_str_slice*	Traits
34	The Deref Trait	*04_traits/07_deref*	Traits
35	The Sized Trait	*04_traits/08_sized*	Traits
36	From and Into	*04_traits/09_from*	Traits
37	Generics vs. Associated Types	*04_traits/10_assoc_vs_generic*	Traits
38	The Clone Trait	*04_traits/11_clone*	Ticket v2
39	The Copy Trait	*04_traits/12_copy*	Ticket v2
40	The Drop Trait	*04_traits/13_drop*	Ticket v2
41	Traits — Wrapping Up	*04_traits/14_outro*	Ticket v2
42	Ticket v2 Introduction	*05_ticket_v2/00_intro*	Ticket v2
43	Enumerations	*05_ticket_v2/01_enum*	Ticket v2
44	Pattern Matching with match	*05_ticket_v2/02_match*	Ticket v2
45	Enum Variants with Data	*05_ticket_v2/03_variants_with_data*	Ticket v2
46	Concise Branching — if let / let else	*05_ticket_v2/04_if_let*	Ticket v2

47	Nullability — Option<T>	*05_ticket_v2/05_nullability*	Ticket v2
48	Fallibility — Result<T, E>	*05_ticket_v2/06_fallibility*	Ticket v2
49	Unwrapping	*05_ticket_v2/07_unwrap*	Ticket v2
50	Error Enums	*05_ticket_v2/08_error_enums*	Ticket v2
51	The Error Trait	*05_ticket_v2/09_error_trait*	Ticket v2
52	Libraries and Binaries	*05_ticket_v2/10_packages*	Ticket v2
53	External Dependencies	*05_ticket_v2/11_dependencies*	Ticket Mgmt
54	The thiserror Crate	*05_ticket_v2/12_thiserror*	Ticket Mgmt
55	TryFrom and TryInto	*05_ticket_v2/13_try_from*	Ticket Mgmt
56	Error::source — Error Chaining	*05_ticket_v2/14_source*	Ticket Mgmt
57	Ticket v2 — Wrapping Up	*05_ticket_v2/15_outro*	Ticket Mgmt
58	Ticket Management Introduction	*06_ticket_management/00_intro*	Ticket Mgmt
59	Arrays	*06_ticket_management/01_arrays*	Ticket Mgmt
60	Vectors	*06_ticket_management/02_vec*	Ticket Mgmt
61	Resizing a Vec	*06_ticket_management/03_resizing*	Ticket Mgmt
62	Iterators	*06_ticket_management/04_iterators*	Ticket Mgmt
63	The .iter() Method	*06_ticket_management/05_iter*	Ticket Mgmt
64	Lifetimes	*06_ticket_management/06_lifetimes*	Ticket Mgmt
65	Iterator Combinators	*06_ticket_management/07_combinators*	Ticket Mgmt
66	impl Trait in Return Position	*06_ticket_management/08_impl_trait*	Ticket Mgmt
67	impl Trait in Argument Position	*06_ticket_management/09_impl_trait_2*	Ticket Mgmt
68	Slices	*06_ticket_management/10_slices*	Ticket Mgmt
69	Mutable Slices	*06_ticket_management/11_mutable_slices*	Threads
70	Type-State Pattern (Two States)	*06_ticket_management/12_two_states*	Threads
71	The Index Trait	*06_ticket_management/13_index*	Threads
72	The IndexMut Trait	*06_ticket_management/14_index_mut*	Threads
73	HashMap	*06_ticket_management/15_hashmap*	Threads
74	BTreeMap and Ordering	*06_ticket_management/16_btreemap*	Threads
75	Threads Introduction	*07_threads/00_intro*	Threads
76	Spawning Threads	*07_threads/01_threads*	Threads
77	The 'static Lifetime Bound	*07_threads/02_static*	Threads
78	Leaking Data	*07_threads/03_leak*	Threads
79	Scoped Threads	*07_threads/04_scoped_threads*	Threads
80	Channels	*07_threads/05_channels*	Threads
81	Interior Mutability	*07_threads/06_interior_mutability*	Threads
82	Two-Way Communication (Acknowledgement)	*07_threads/07_ack*	Threads

83	A Dedicated Client Type	*07_threads/08_client*	Threads
84	Bounded vs. Unbounded Channels	*07_threads/09_bounded*	Async Rust
85	Patch Updates	*07_threads/10_patch*	Async Rust
86	Locks — Mutex	*07_threads/11_locks*	Async Rust
87	Readers and Writers — RwLock	*07_threads/12_rw_lock*	Async Rust
88	Shared-State Concurrency Without Channels	*07_threads/13_without_channels*	Async Rust
89	The Sync Trait	*07_threads/14_sync*	Async Rust
90	Async Rust Introduction	*08_futures/00_intro*	Async Rust
91	Asynchronous Functions	*08_futures/01_async_fn*	Async Rust
92	Spawning Async Tasks	*08_futures/02_spawn*	Section
93	Runtime Architecture	*08_futures/03_runtime*	Section
94	The Future Trait	*08_futures/04_future*	Section
95	Don't Block the Async Runtime	*08_futures/05_blocking*	Section
96	Async-Aware Primitives	*08_futures/06_async_aware_primitives*	Section
97	Cancellation	*08_futures/07_cancellation*	Section
98	Async Rust — Outro	*08_futures/08_outro*	Section